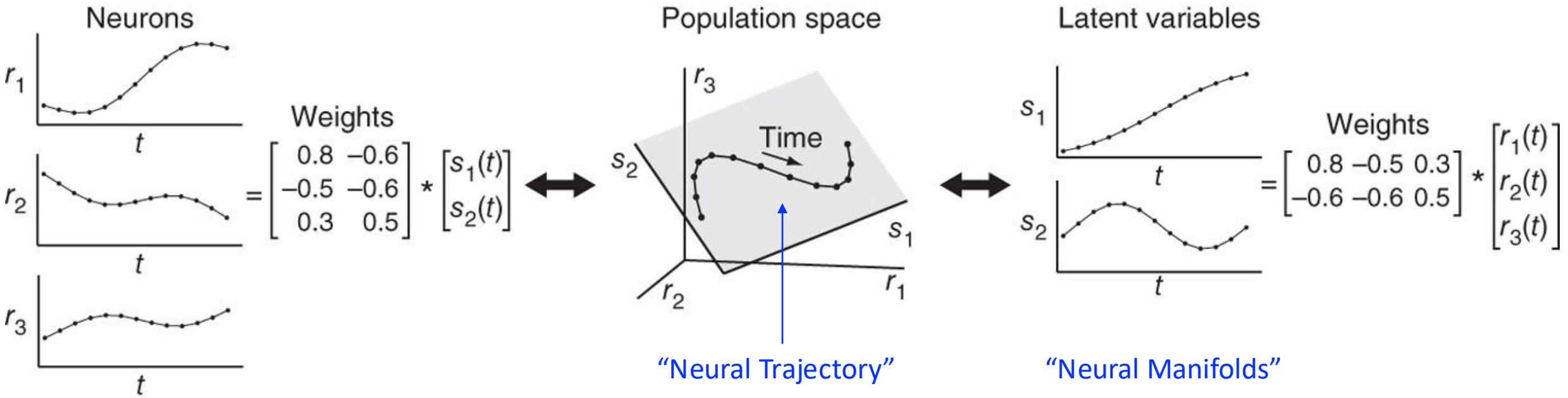
Unsupervised Learning

Outline

- Unsupervised learning for dimensionality reduction
- Unsupervised learning for data clustering

Unsupervised learning for dimensionality reduction

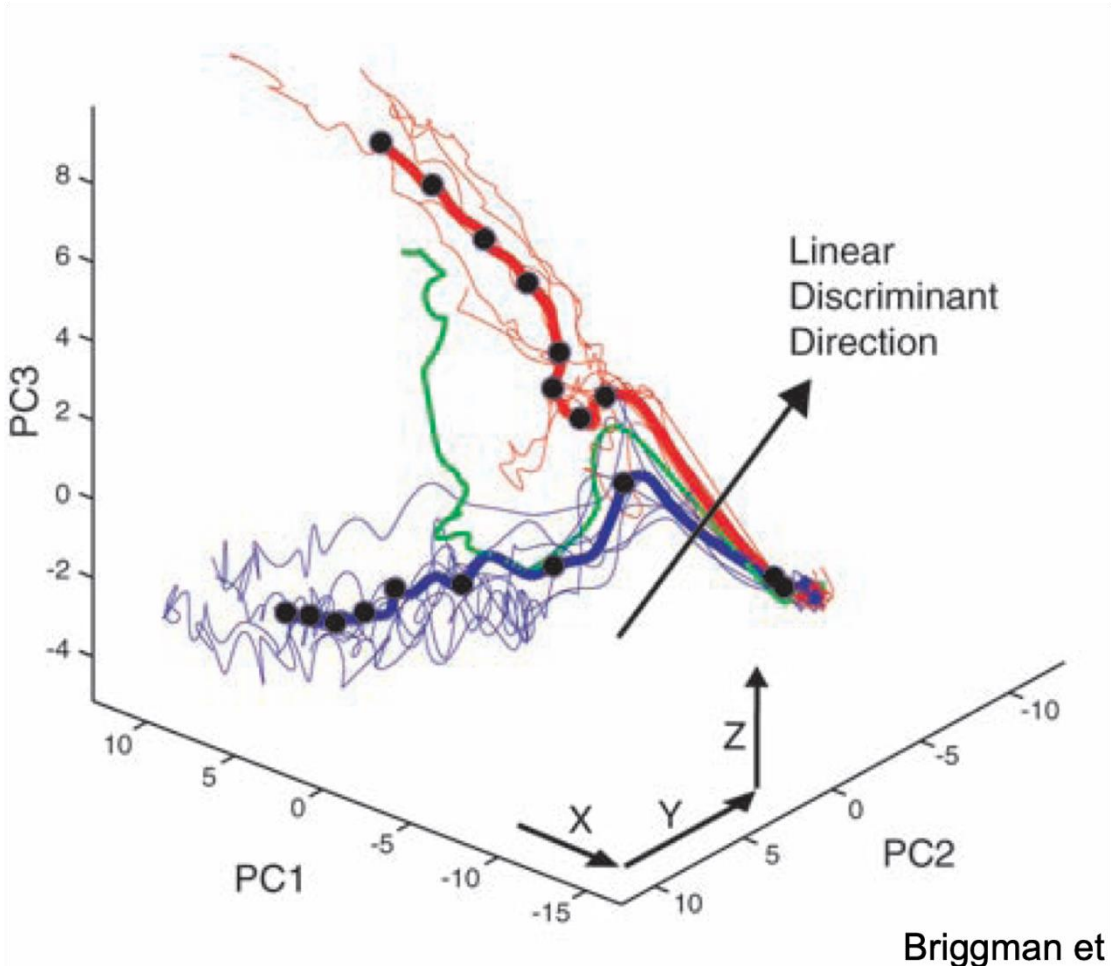
Why dimensionality reduction?



Cunningham and Yu, 2014

Why dimensionality reduction?

- 143 leech ganglia cells
- Swimming (blue) vs Crawling (red)
- Vacillating before settling down (green)

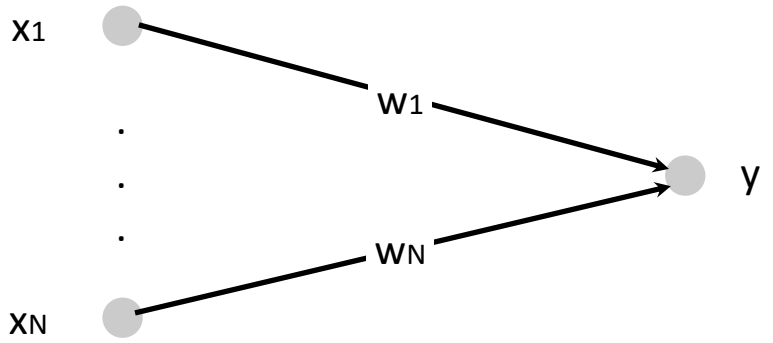


Briggman et al. Science 2005

Principal Component Analysis

Principal Component Analysis (PCA)

- We aim to find a linear projection (vector) that preserves the most information of the data
  - We need to define what preserves *the most information*
- Perhaps, we may keep substantial data information if we try to preserve the **variance** of the data



Find $w = [w_1, .. , w_N]$ to maximize the variance of y

Find a direction (axis) in which the most information about the following shape is preserved.

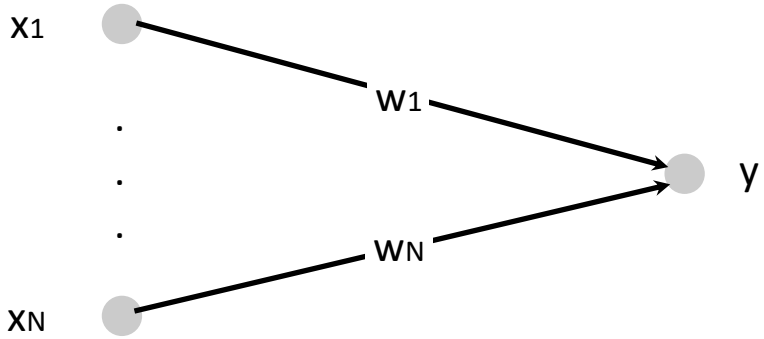






Such axes are often called a principal axis.

How can we preserve variance?



Find $\mathbf{w} = [w_1, .. , w_N]$ to maximize the variance of y

$$y = \boldsymbol{w}^T \boldsymbol{x}$$

Assume $E[\boldsymbol{x}] = \mathbf{0}$ $\longrightarrow$ $E[y] = E[\boldsymbol{w}^T \boldsymbol{x}] = \boldsymbol{w}^T E[\boldsymbol{x}] = 0$

$$\text{Var}[y] = E[(y - E[y])^2] = E[y^2]$$

$$= E[(\boldsymbol{w}^T \boldsymbol{x})^2] = E[\boldsymbol{w}^T \boldsymbol{x}(\boldsymbol{w}^T \boldsymbol{x})^T] = \boldsymbol{w}^T E[\boldsymbol{x}\boldsymbol{x}^T]\boldsymbol{w}$$

$E[\boldsymbol{x}\boldsymbol{x}^T] \equiv \boldsymbol{R}$ ~ covariance matrix of **x**

$$\text{Var}[y] = \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w}$$

How can we preserve variance?

Find w that maximizes: $\text{Var}[y] = \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w}$

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{argmax}\, \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w}$$

But $k\mathbf{w}$ for any $k > 0$ can satisfiy this $\longrightarrow$ $\|\boldsymbol{w}\| = 1$

So, we have a contrainsed optimzation problem

$$\boldsymbol{w}^* = \underset{\boldsymbol{w}}{argmax}\, \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w} - \boldsymbol{\lambda} \boldsymbol{w}^T \boldsymbol{w}$$

$$\frac{\partial \mathbf{w}^T R\mathbf{w} - \lambda \mathbf{w}^T \mathbf{w}}{\partial \mathbf{w}} = R\mathbf{w} - \lambda \mathbf{w} = 0$$

$$R\mathbf{w} = \lambda \mathbf{w}$$

w is the eigenvector of R with the eigenvalue of λ

Principal Components

- *y* = principal components (PCs)
- Variance of y = eigenvalue corresponding to eigenvector used for *y*

$y = \boldsymbol{w}^T \boldsymbol{x}$ → project $\boldsymbol{x}$ onto *y* with $\boldsymbol{w}$

$\text{Var}[y] = \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w} = \boldsymbol{w}^T (\boldsymbol{\lambda} \boldsymbol{w}) = \boldsymbol{\lambda} \boldsymbol{w}^T \boldsymbol{w} = \boldsymbol{\lambda} \|\boldsymbol{w}\|^2 = \boldsymbol{\lambda}$

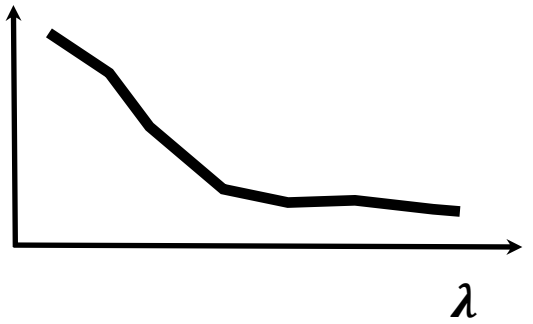
Principal Components

- *y* = principal components (PCs)
- Variance of y = eigenvalue corresponding to eigenvector used for *y*

$$y = \boldsymbol{w}^T \boldsymbol{x} \quad \rightarrow \text{project } \boldsymbol{x} \text{ onto } y \text{ with } \boldsymbol{w}$$

$$\text{Var}[y] = \boldsymbol{w}^T \boldsymbol{R} \boldsymbol{w} = \boldsymbol{w}^T (\boldsymbol{\lambda} \boldsymbol{w}) = \boldsymbol{\lambda} \boldsymbol{w}^T \boldsymbol{w} = \boldsymbol{\lambda} \|\boldsymbol{w}\|^2 = \boldsymbol{\lambda}$$

- How many PCs should we find?
  - Sort out the eigenvalues
  - Start from the largest eigenvalues
  - Compute the cumulative sum of eigenvalues
  - Stop when the cumulative sum reaches a certain level (e.g. 80%)

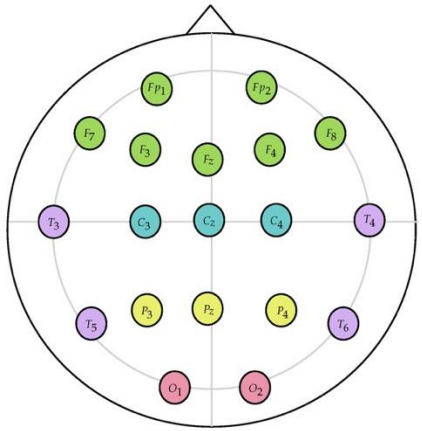


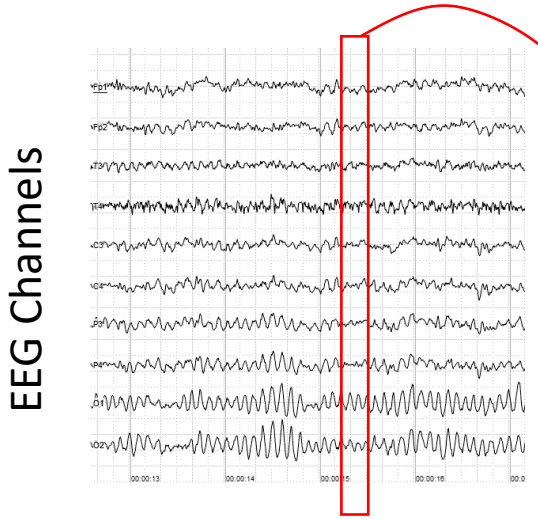
Principal Components

- PCA is an unsupervised method: no need to know class (or event) info
- PCs are uncorrelated with each other
  - Eigenvectors are orthogonal to each other: $\boldsymbol{w}_i^T \boldsymbol{w}_j = 0, i \neq j$

Principal Components

- PCA is an unsupervised method: no need to know class (or event) info
- PCs are uncorrelated with each other
  - Eigenvectors are orthogonal to each other: $\boldsymbol{w}_i^T \boldsymbol{w}_j = 0, i \neq j$
- PC of a multivariate time series is a new time series

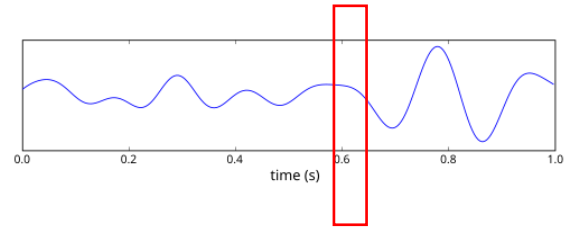




$\boldsymbol{x_t} \in \mathbb{R}^{\boldsymbol{C}}$

PCA
$\boldsymbol{w}$

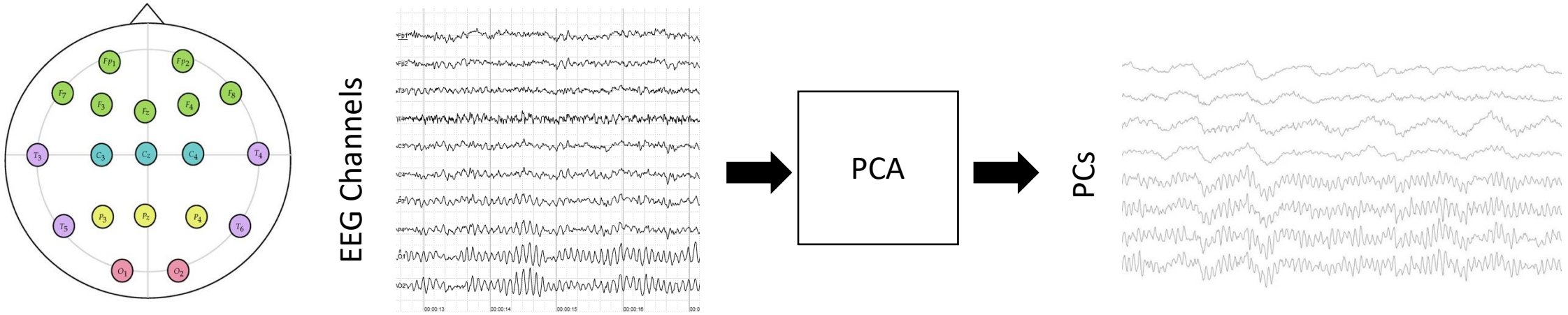
$y_t = \boldsymbol{w}^T \boldsymbol{x}_t$



C = # channels

Principal Components

- PCA is an unsupervised method: no need to know class (or event) info
- PCs are uncorrelated with each other
  - Eigenvectors are orthogonal to each other: $\boldsymbol{w}_i^T \boldsymbol{w}_j = 0, i \neq j$
- PC of a multivariate time series is a new time series

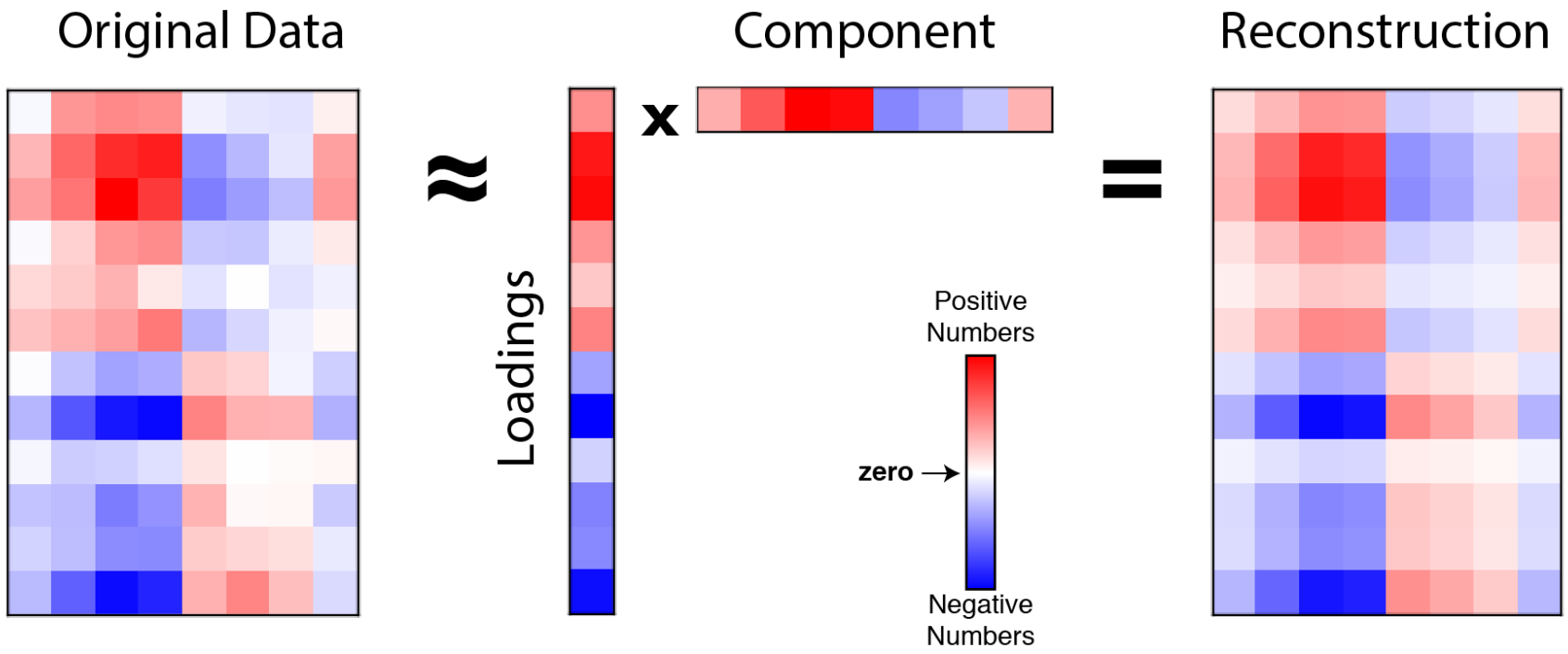


Principal Components

- PCA is an unsupervised method: no need to know class (or event) info
- PCs are uncorrelated with each other
  - Eigenvectors are orthogonal to each other: $\boldsymbol{w}_i^T \boldsymbol{w}_j = 0, i \neq j$
- PC of a multivariate time series is a new time series
- Original data can be perfectly reconstructed by all PCs

Principal Components

- Reconstruction of data using a PC

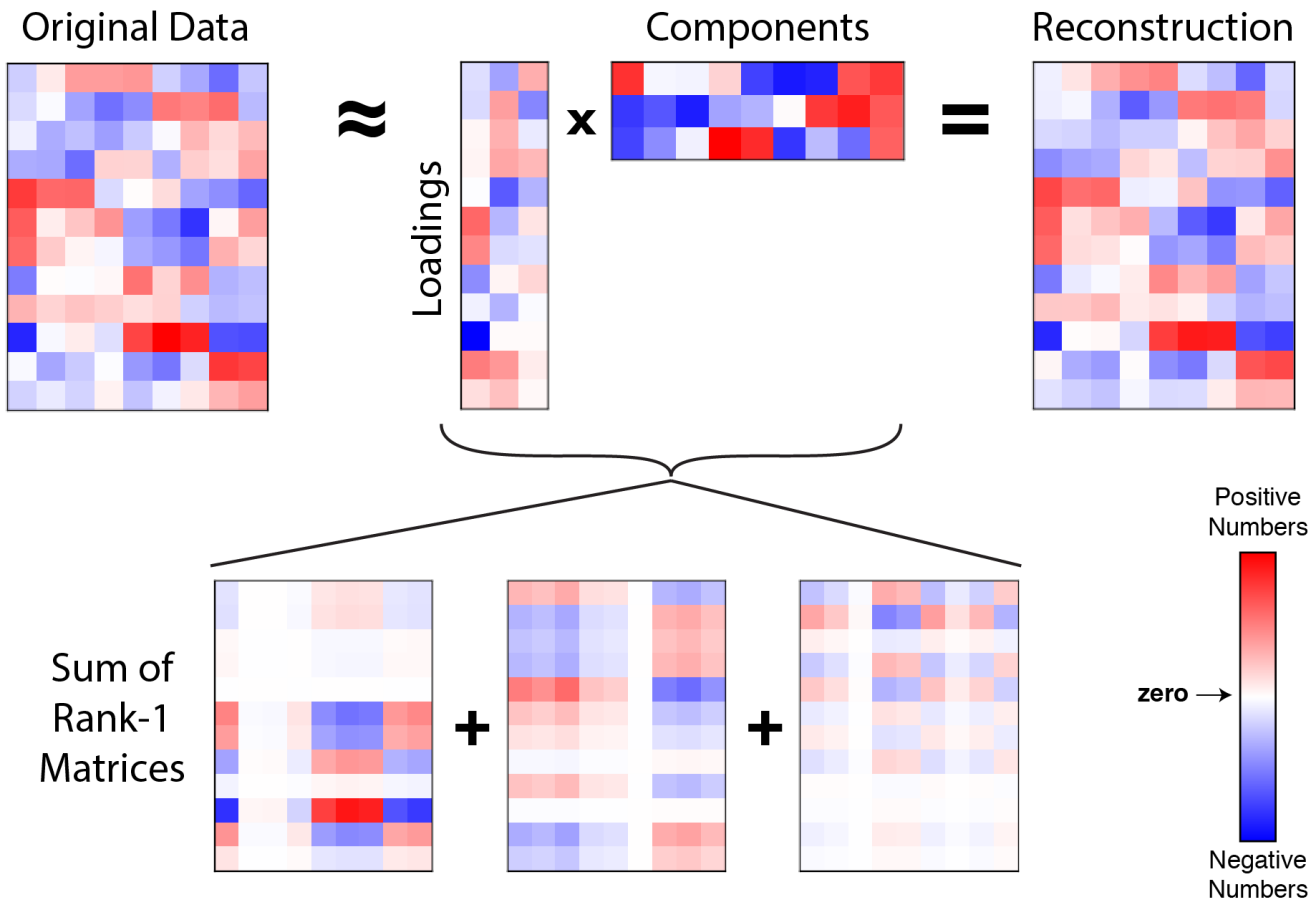


*Loadings = Elements of eigenvector

(Courtesy: Alex Williams)

Principal Components

- Reconstruction of data using PCs



(Courtesy: Alex Williams)

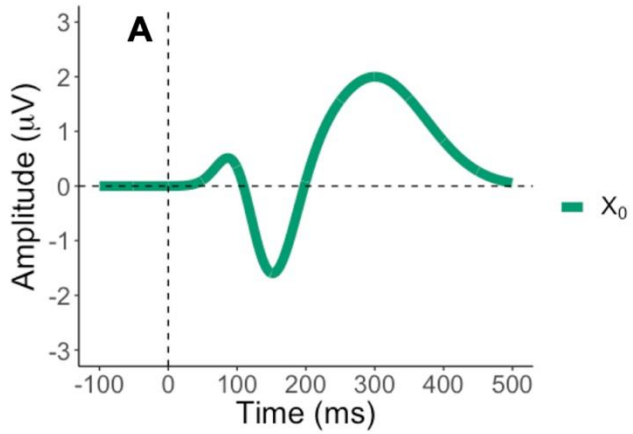
Principal Components

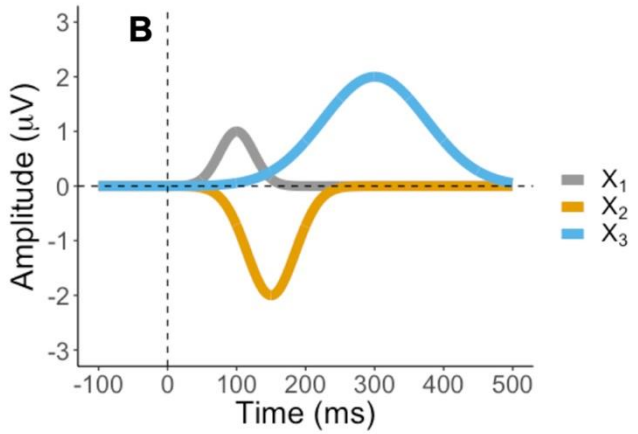
- PCA is an unsupervised method: no need to know class (or event) info
- PCs are uncorrelated with each other
  - Eigenvectors are orthogonal to each other: $\boldsymbol{w}_i^T \boldsymbol{w}_j = 0, i \neq j$
- PC of a multivariate time series is a new time series
- Original data can be perfectly reconstructed by all PCs
- PCA helps find covariance structures
    - Temporal PCA: finding temporal covariance of a time series

Temporal PCA for ERP data

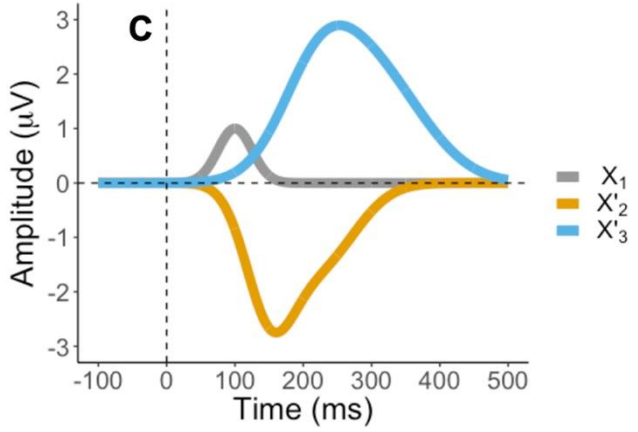
- Observed ERP = a combination of underlying signals (time series)

Observed ERP





Candidate set 1

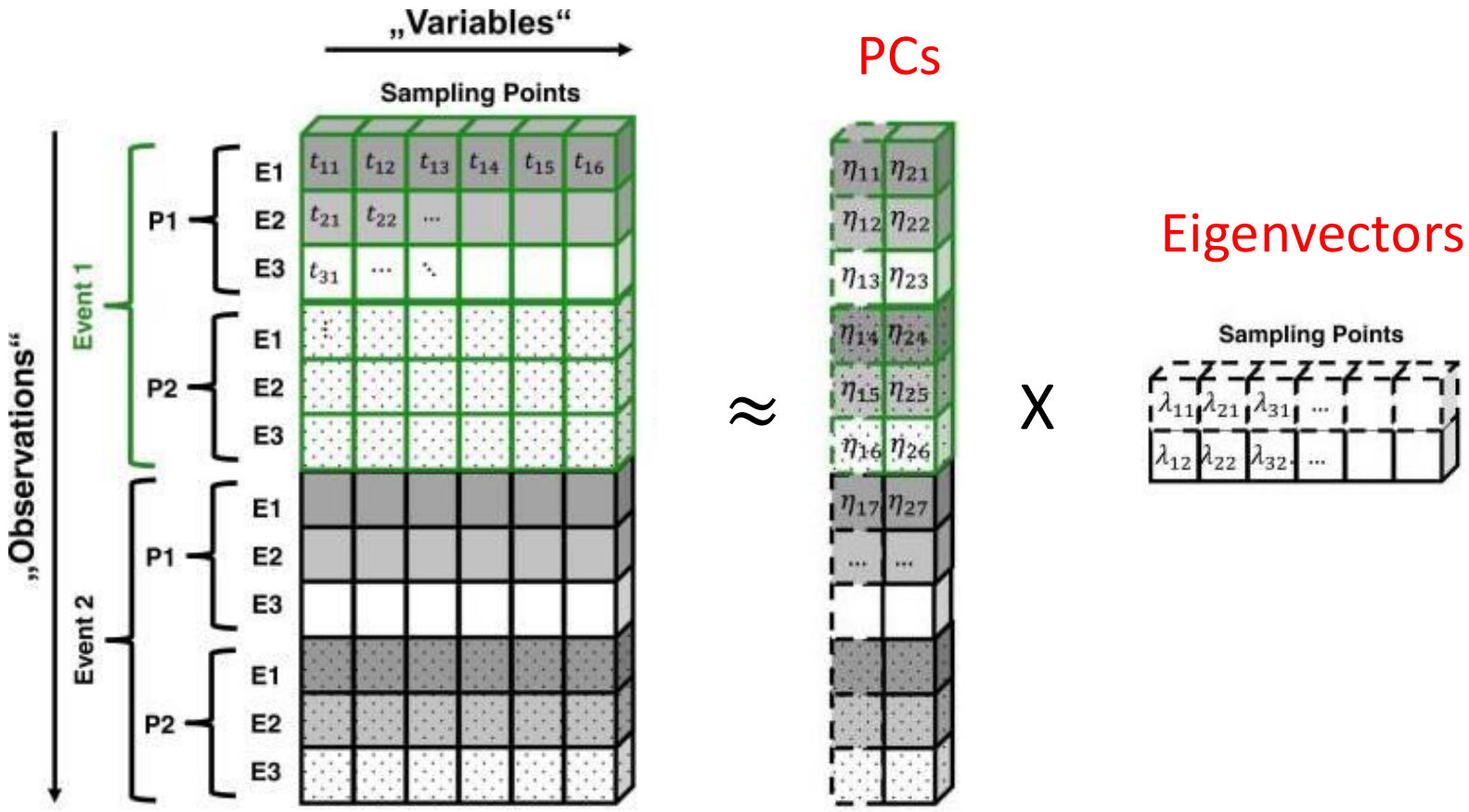


Candidate set 2

(McWeeny and Norton, 2021)

Temporal PCA for ERP data

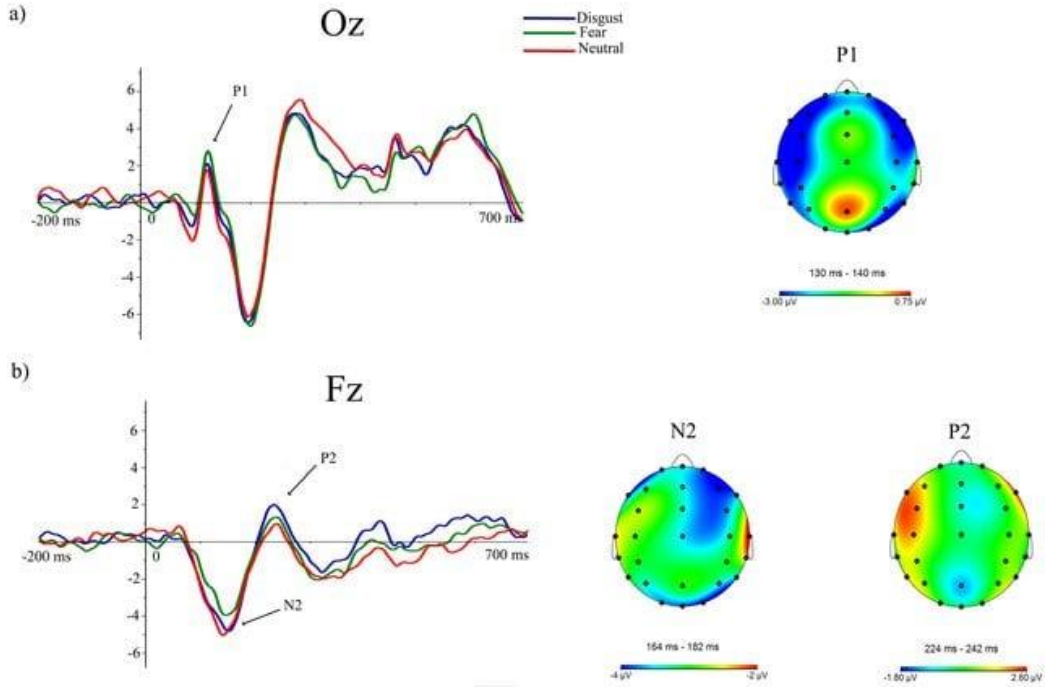
- Temporal PCA aims to find latent vectors with similar pattern across participants, electrodes and conditions

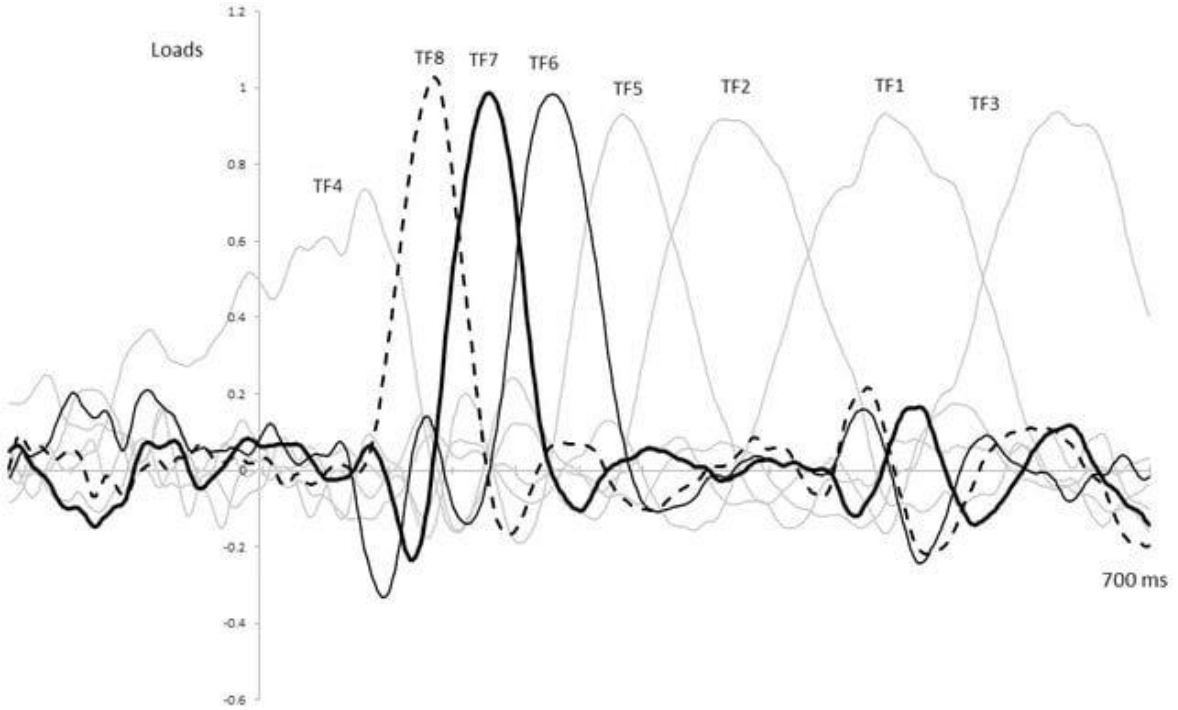


P: participant, E: electrode

(Scharf et al., 2022)

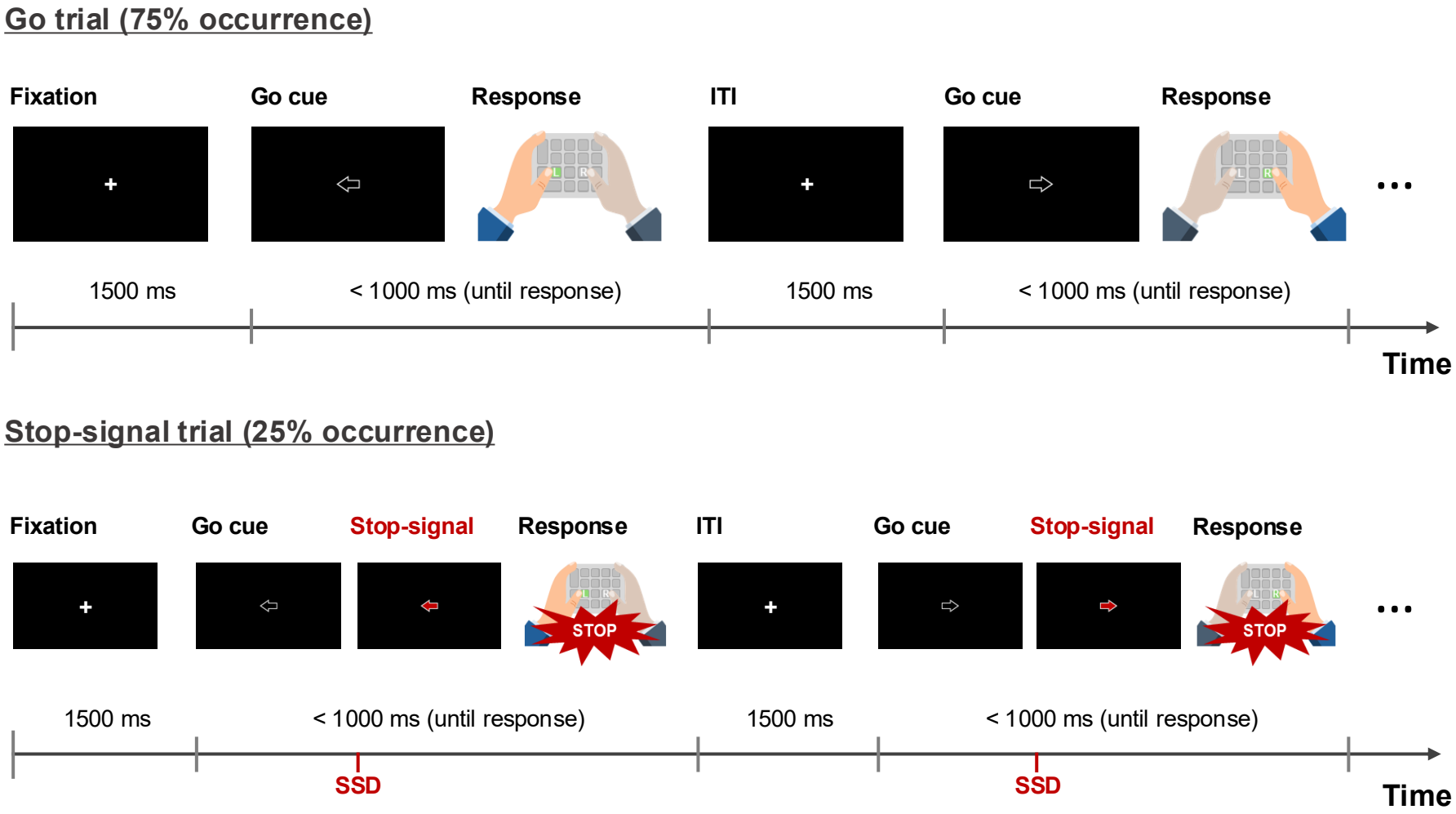
Temporal PCA for ERP data



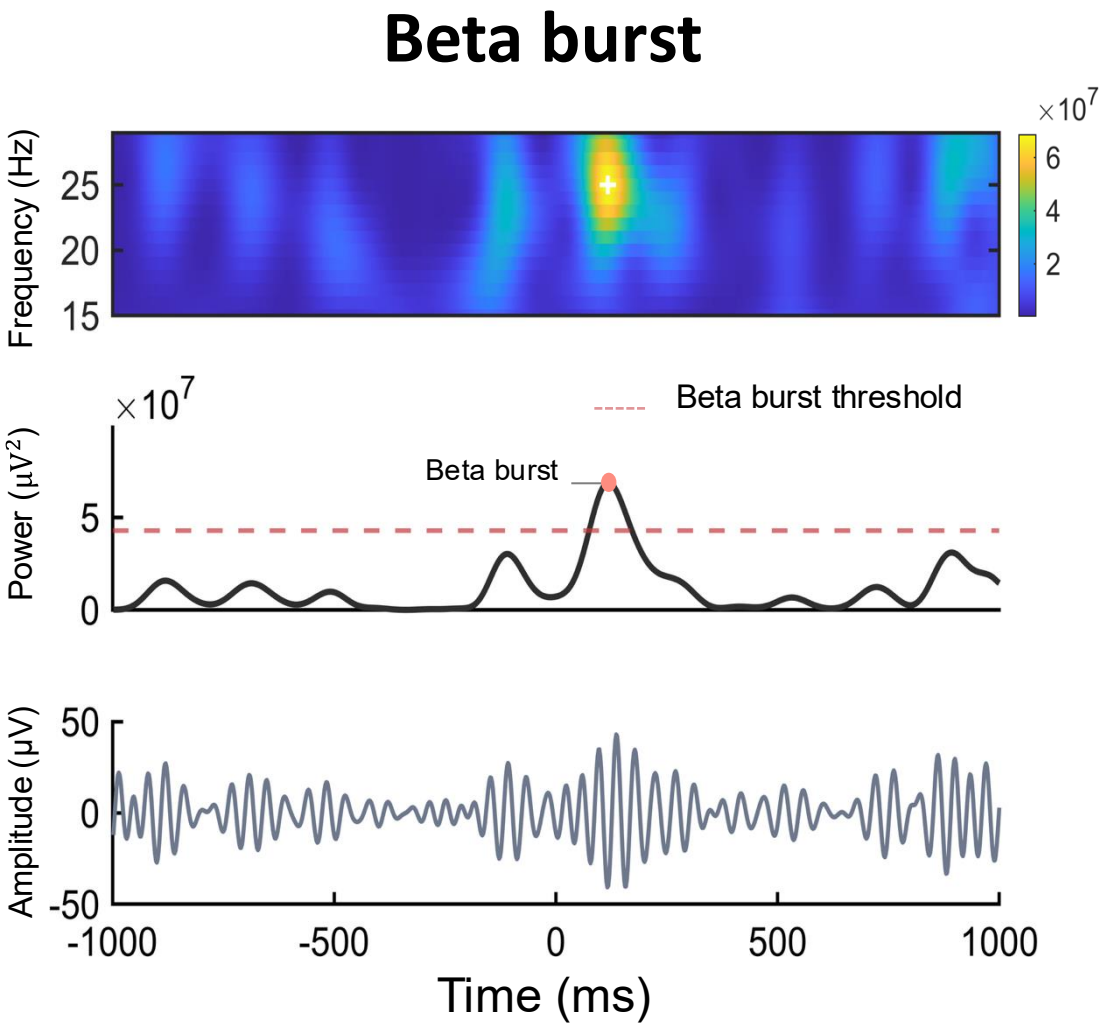


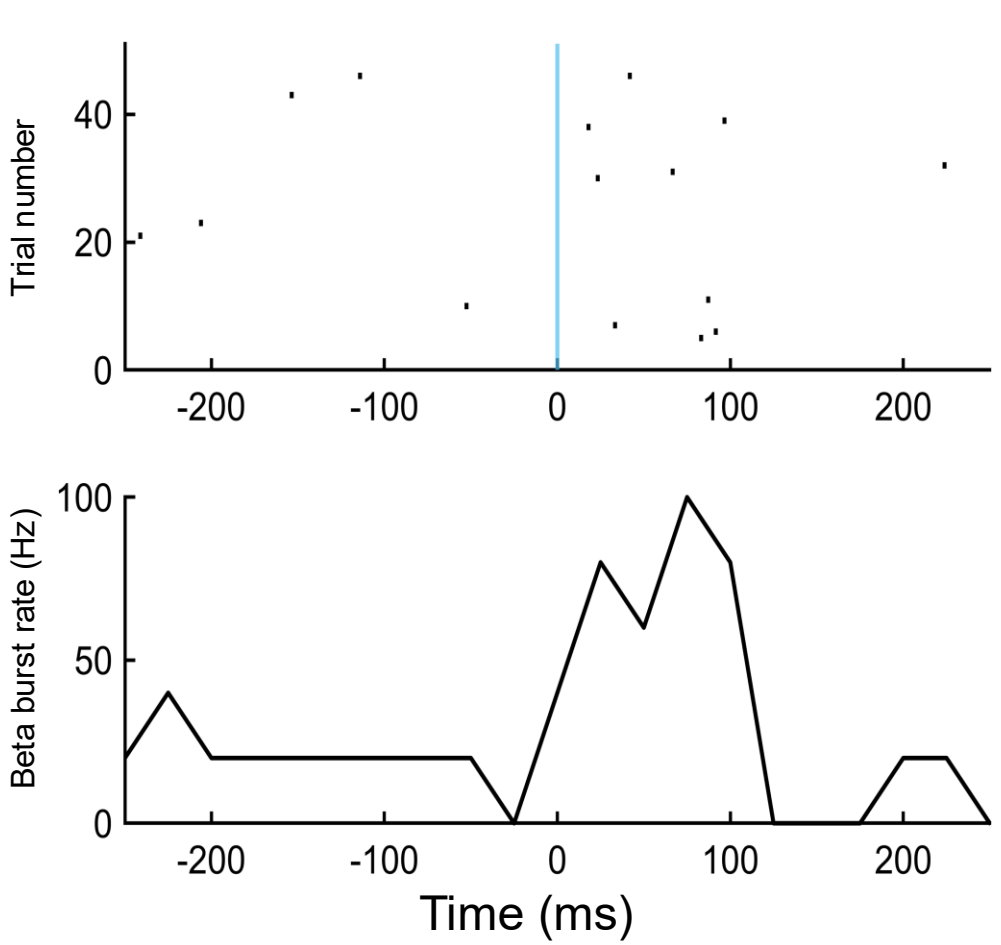
(Ruiz-Padial and Mercado., 2021)

Case Study (Oh et al., 2026)

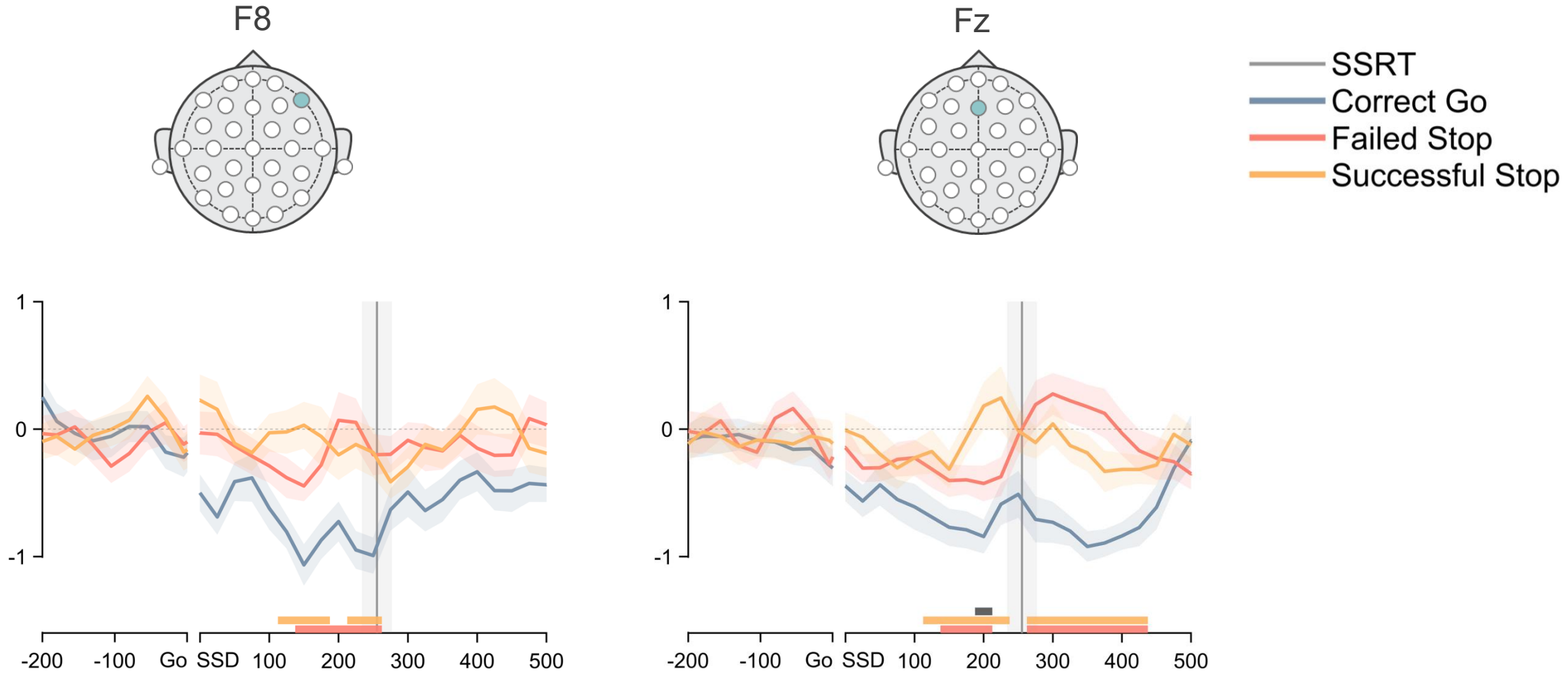


Case Study (Oh et al., 2026)



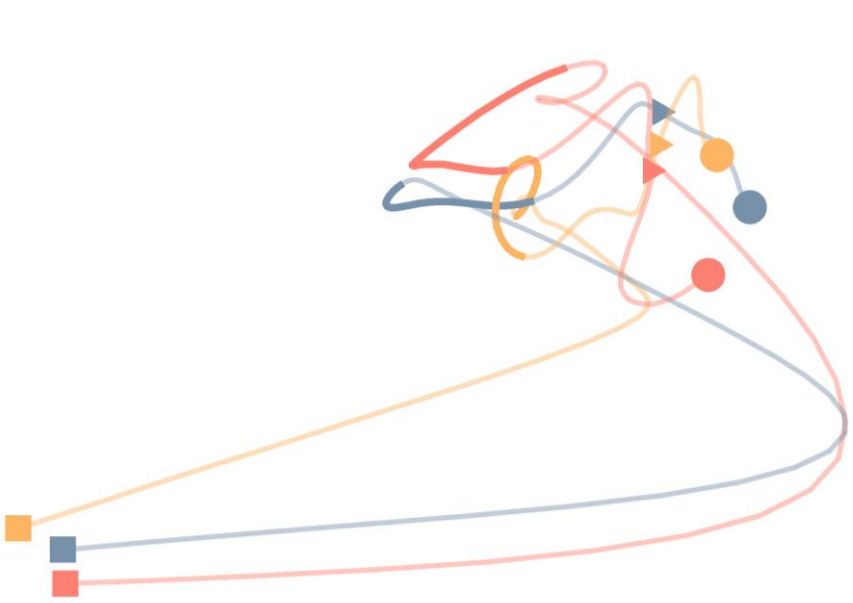


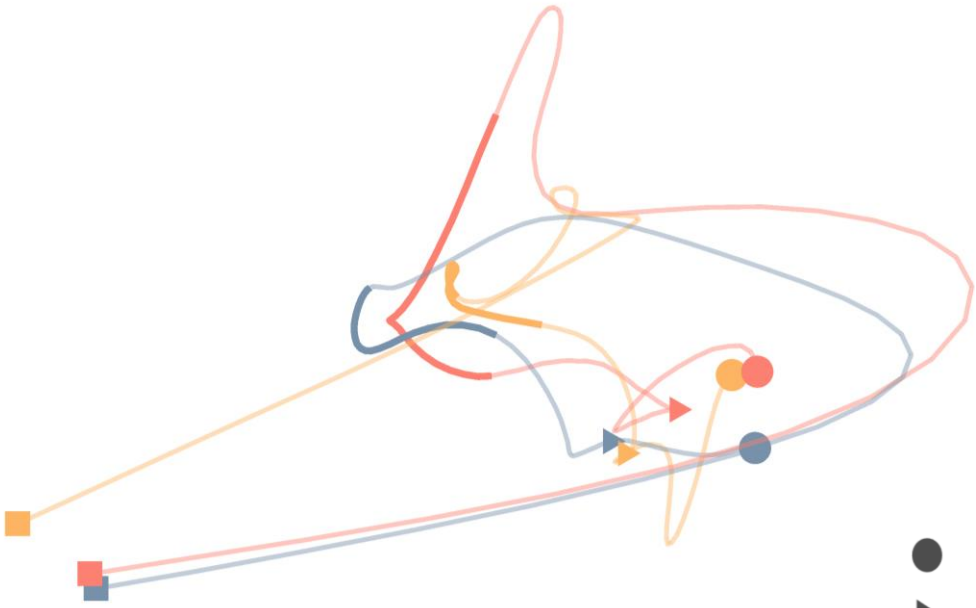
Case Study (Oh et al., 2026)

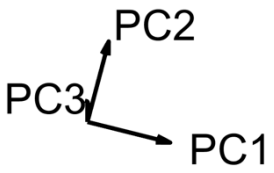


Case Study (Oh et al., 2026)









Fixation
Go-cue
Trial end

Correct Go
Failed Stop
Successful Stop

t-SNE

t-distributed Stochastic Neighbor Embedding

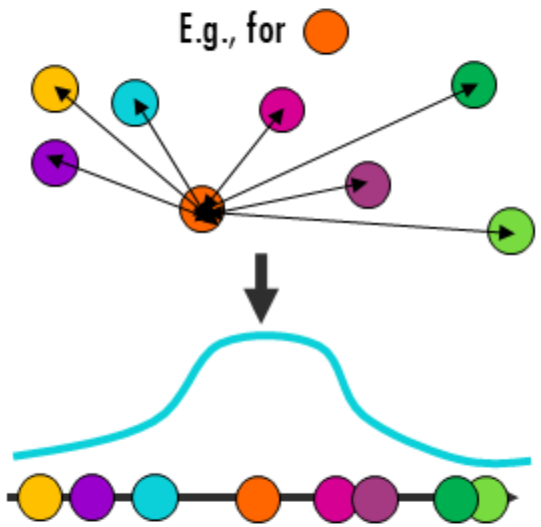
(BiostatSQUID)

How does t-SNE summarise many dimensions into 2?

1

Measure pairwise similarities

E.g., for



Distance between cells

High similarity score

Low similarity score

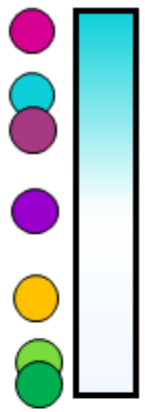
...

for all pairs of cells in the dataset

2

Create probabilities

High probability of being neighbour of



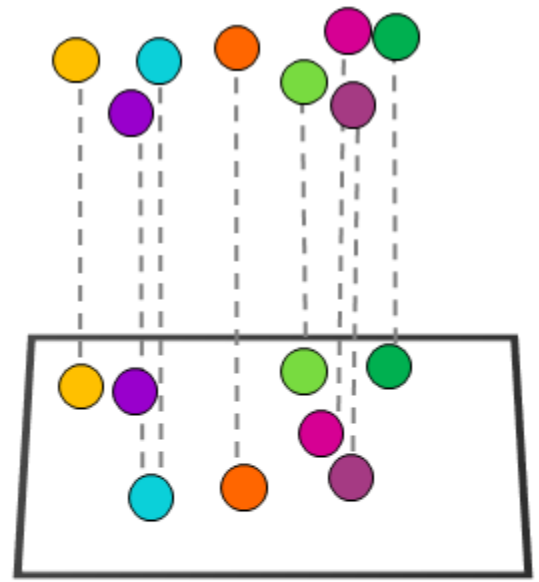
Low probability of being neighbour of

for all the cells in the dataset

3

Map to lower dimensions

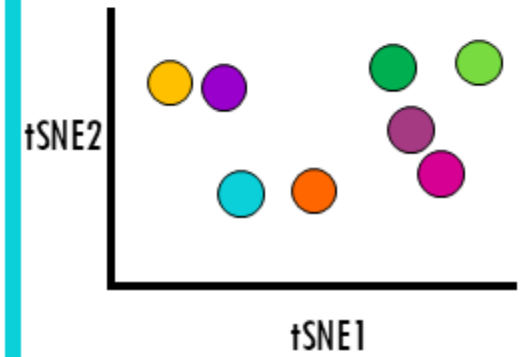
Multi-D



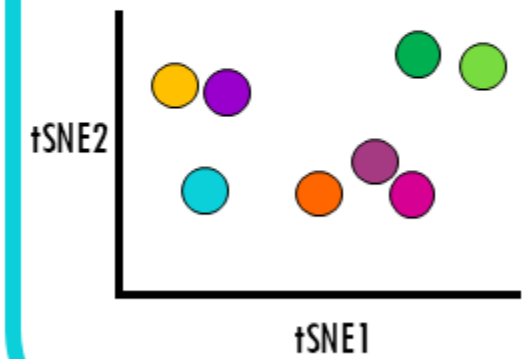
2D

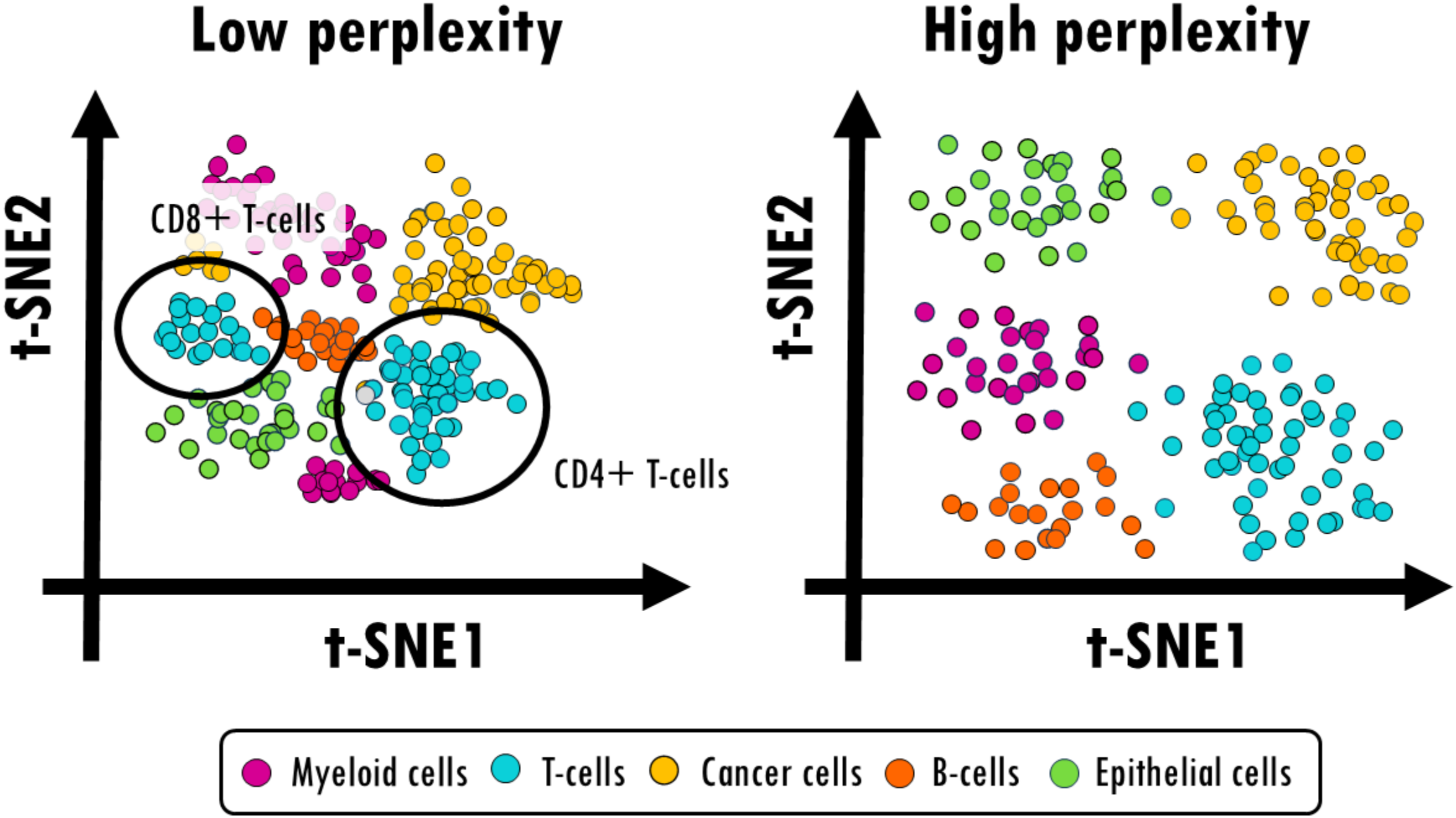
4

Optimise the layout



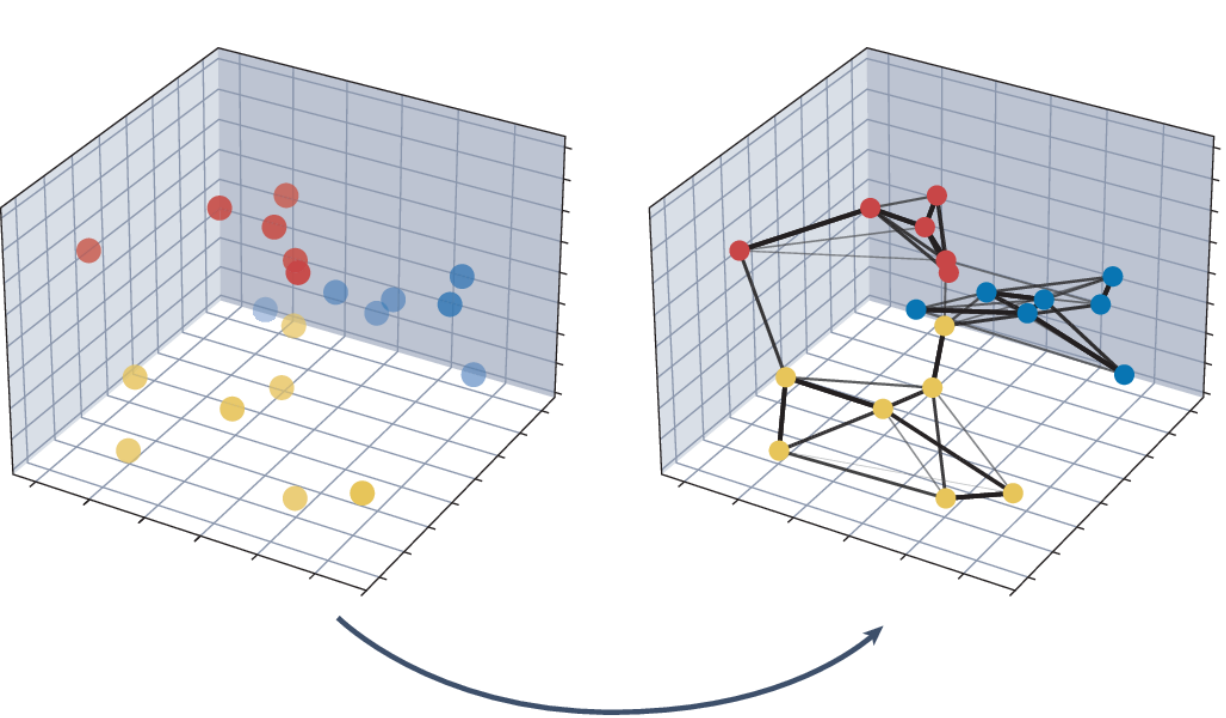
Gradient descent



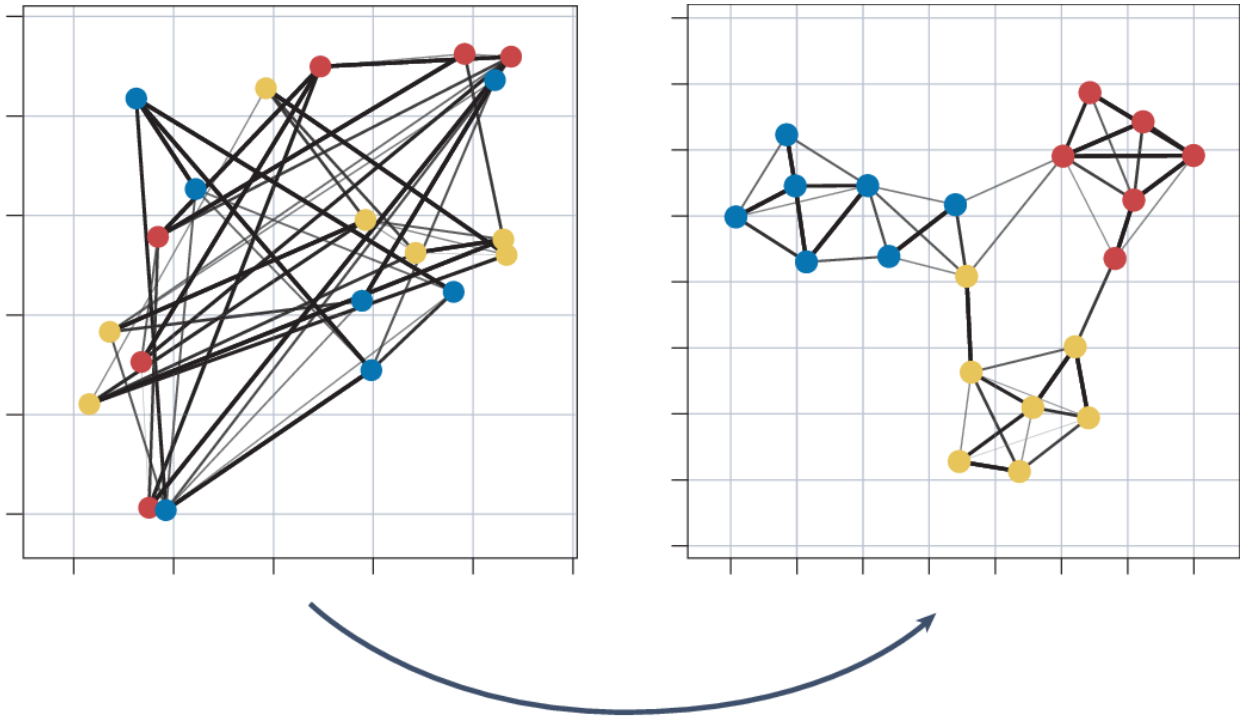

Low perplexity
High perplexity
t-SNE2
t-SNE1
CD8+ T-cells
CD4+ T-cells
Myeloid cells
T-cells
Cancer cells
B-cells
Epithelial cells

UMAP

Uniform Manifold Approximation and Projection



Build a topological representation of
the high-dimensional data



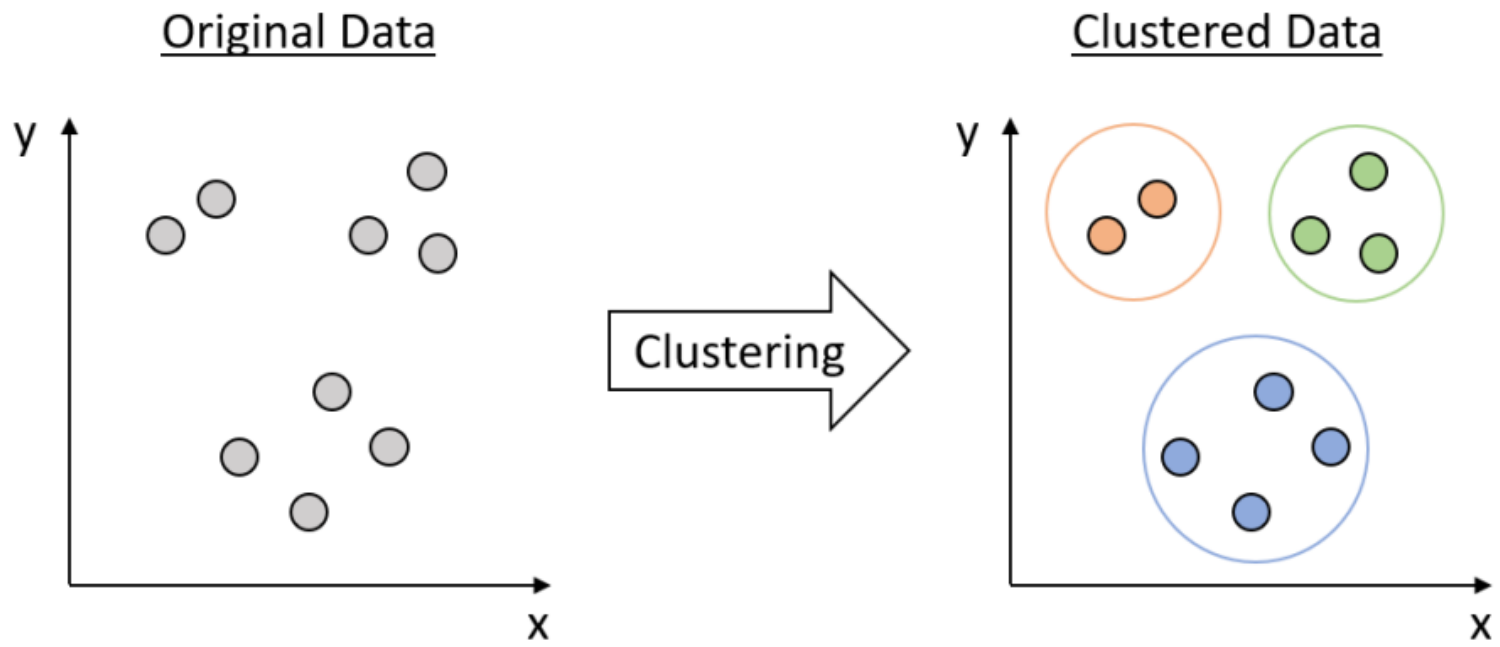
Optimize a low-dimensional embedding
to have similar topological representation

(Healy and McInnes, 2024)

Feature	t-SNE	UMAP
Speed	Slower, computationally intensive	Faster, more memory-efficient
Scalability	Less suitable for large datasets	Suitable for large and small datasets
Consistency	Results can vary between runs due to its stochastic nature	Generally more consistent across runs
Main Use Case	Detailed exploration of local clusters, especially in well-separated datasets	Visualizing overall data organization and connections between clusters

Unsupervised learning for data clustering

Clustering



K-means algorithm

Data: $\{\boldsymbol{x_1}, \boldsymbol{x_2}, \ldots, \boldsymbol{x_N}\}, \boldsymbol{x_i} \in \mathbb{R}^{\boldsymbol{D}}$

Goal: partition the data into K clusters

Method: Use distance between data points

Assignment

Data: $\{\boldsymbol{x_1}, \boldsymbol{x_2}, \ldots, \boldsymbol{x_N}\}, \boldsymbol{x_i} \in \mathbb{R}^{\boldsymbol{D}}$

Let $\boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \ldots, \boldsymbol{\mu_K}$ be the centers of each cluster

Indicator variables, $r_{n,k} \in \{0, 1\}$

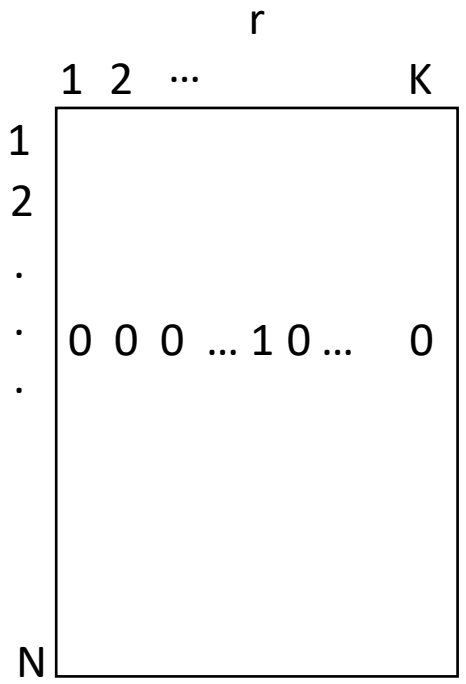
n: index of data point $\boldsymbol{x}_n$

k: index of cluster, k = 1, …, K

If $\boldsymbol{x}_n$ belongs to a cluster k,

$$r_{n,j} = \begin{cases} 1 & if\ j = k \\ 0 & if\ j \neq k \end{cases}$$

1-of-K coding scheme



Distortion measure

Objective function (*J*): the sum of squares of the distance from each data point to its center

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

Objective: find $r_{n,k}$ and $\boldsymbol{\mu}_k$ to minimize *J*

Learning

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

Step 1.

Start with random $\boldsymbol{\mu}_k$

Fix $\boldsymbol{\mu}_k$ and minimize *J* with respect to $r_{n,k}$ → called expectation step (E-step)

Step 2.

Fix $r_{n,k}$ and minimize *J* with respect to $\boldsymbol{\mu}_k$ → called maximization step (M-step)

Repeat steps 1 and 2 until convergence

Learning

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

E-step: $r_{n,k}$

Objective function *J* is a linear function of $r_{n,k}$

$\boldsymbol{x}_n$'s are independent

So, for each n, find $r_{n,k}$ such that $\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$ is minimum

$\Leftrightarrow$ find the closest $\boldsymbol{\mu}_k$ to $\boldsymbol{x}_n$

$$\therefore r_{n,k} = \begin{cases} 1 & if\ k = \underset{j}{\operatorname{argmin}} \|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2 \\ 0 & otherwise \end{cases}$$

Learning

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$$

M-step: $\boldsymbol{\mu}_k$

Objective function *J* is a quadratic function of $\boldsymbol{\mu}_k$

$$\frac{\partial J}{\partial \boldsymbol{\mu}_k} = -2 \sum_{n=1}^{N} r_{n,k} (\boldsymbol{x}_n - \boldsymbol{\mu}_k) = 0 \qquad \text{for } r_{n,k} = 1$$

$$\boldsymbol{\mu}_k \sum_{n=1}^{N} r_{n,k} = \sum_{n=1}^{N} r_{n,k} \boldsymbol{x}_n$$

$\sum_{n=1}^{N} r_{n,k}$ = # data points in cluster k

$$\boldsymbol{\mu}_k = \frac{\sum_{n=1}^{N} r_{n,k} \boldsymbol{x}_n}{\sum_{n=1}^{N} r_{n,k}}$$

So, $\boldsymbol{\mu}_k$ is the mean of all $\boldsymbol{x}_n$ belonging to k-th cluster

$\Leftrightarrow$ k-means

Learning

Convergence: repeat E-step and M-step until no change in assignment occurs

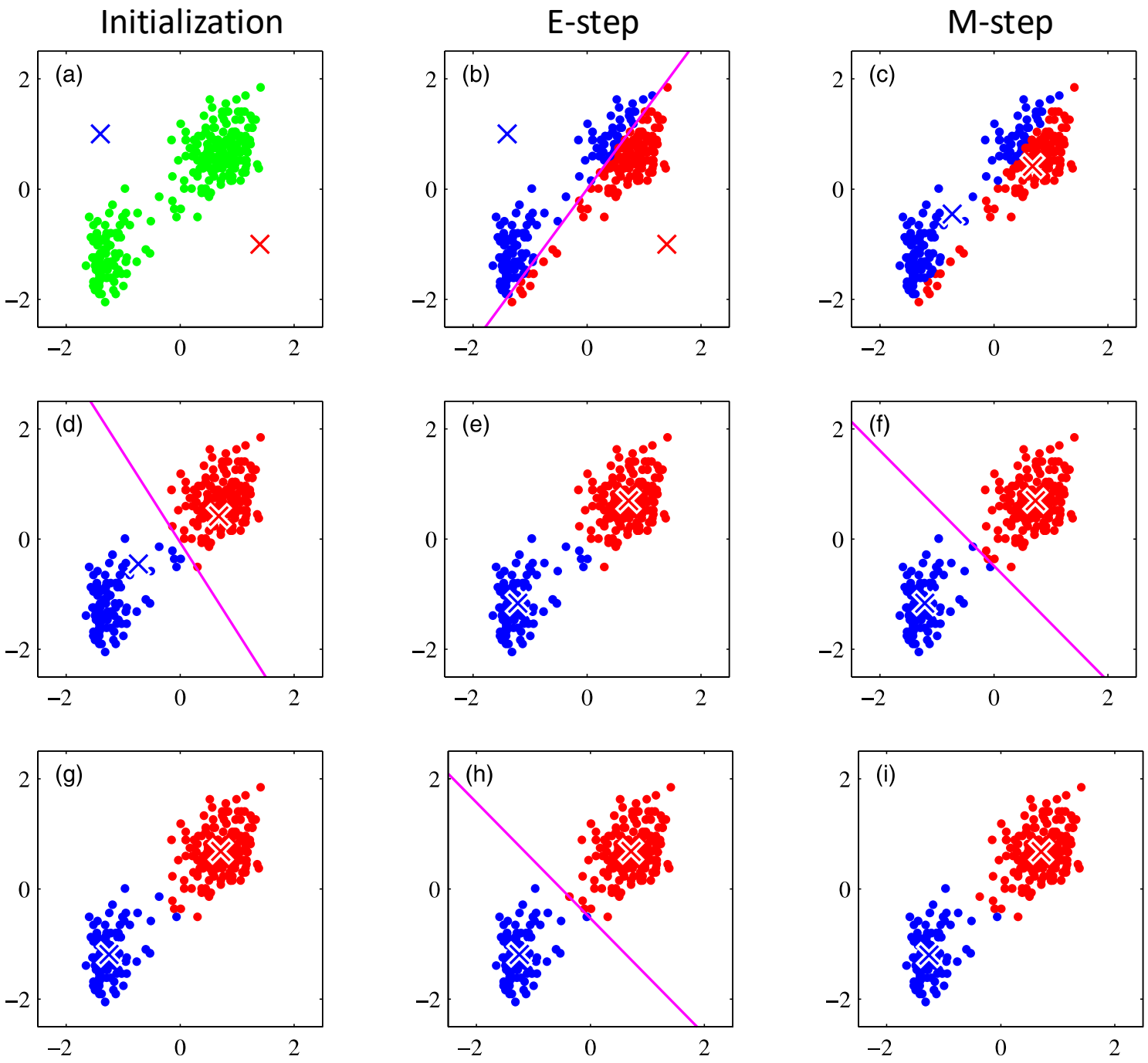
Initialization

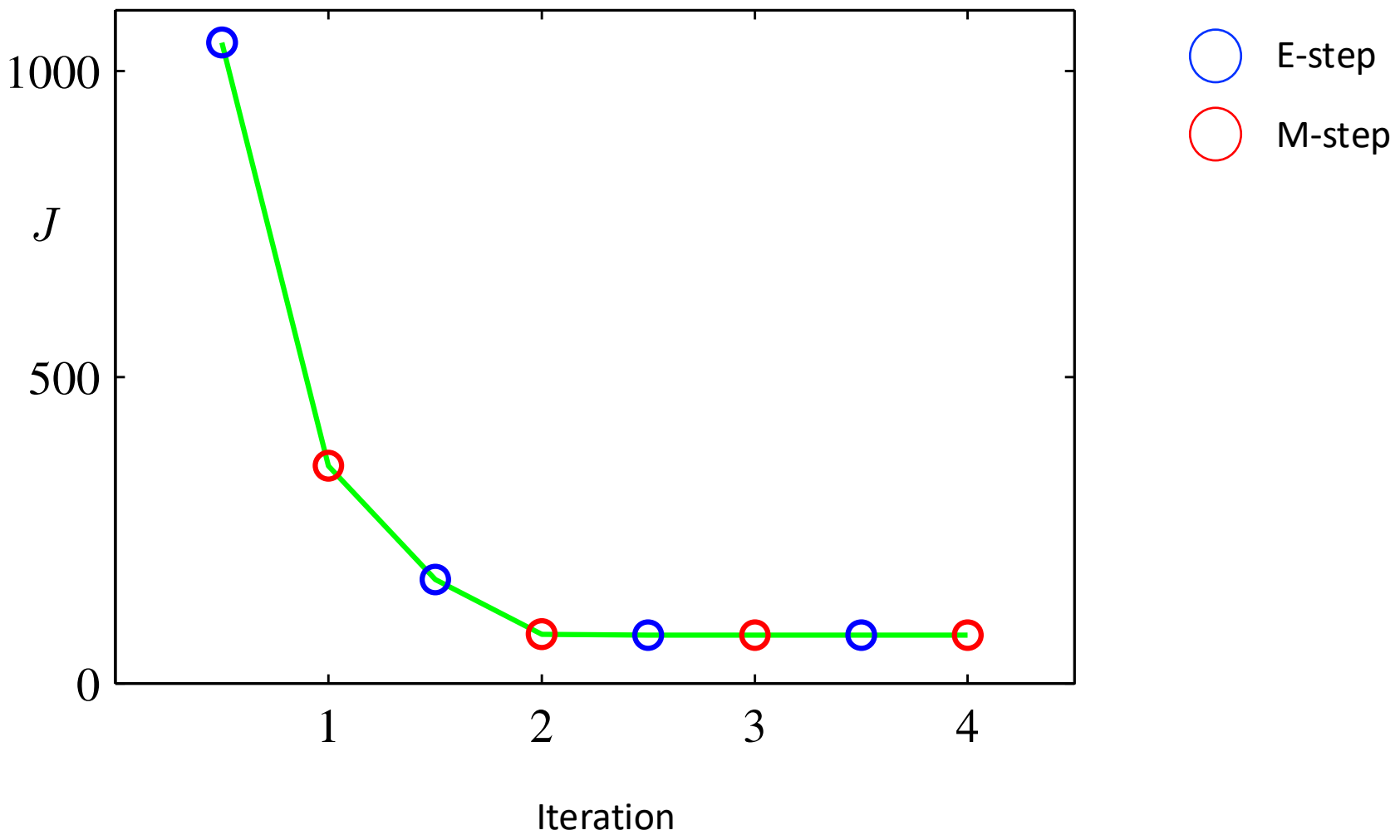
How to initialize $\boldsymbol{\mu}_k$?

1) K random vectors within the input space

2) $\boldsymbol{\mu}_k = \boldsymbol{x}_n$ for some randomly selected *n*

K-means algorithm is often used to initialize centers for other models (e.g., Gaussian mixture models)


Initialization
E-step
M-step
(a)
(b)
(c)
(d)
(e)
(f)
(g)
(h)
(i)
2
0
−2
−2
0
2


1000
J
500
0
1
2
3
4
Iteration
E-step
M-step

On-line algorithm

Update of $\boldsymbol{\mu}_k$ on a sample-by-sample basis

$$\boldsymbol{\mu}_k{}^{\boldsymbol{NEW}} = \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}} - \boldsymbol{\eta}\frac{\partial J}{\partial \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}}} \qquad \boldsymbol{\eta} \text{ is a learning rate}$$

$$\frac{\partial J}{\partial \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}}} = -2\left(\boldsymbol{x}_n - \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}}\right) \quad \text{at } n \qquad \rightarrow \text{stochastic estimate of gradient}$$

So,

$$\boldsymbol{\mu}_k{}^{\boldsymbol{NEW}} = \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}} + \boldsymbol{\eta}\left(\boldsymbol{x}_n - \boldsymbol{\mu}_k{}^{\boldsymbol{OLD}}\right)$$

Often, $\boldsymbol{\eta}$ is monotonically decreasing

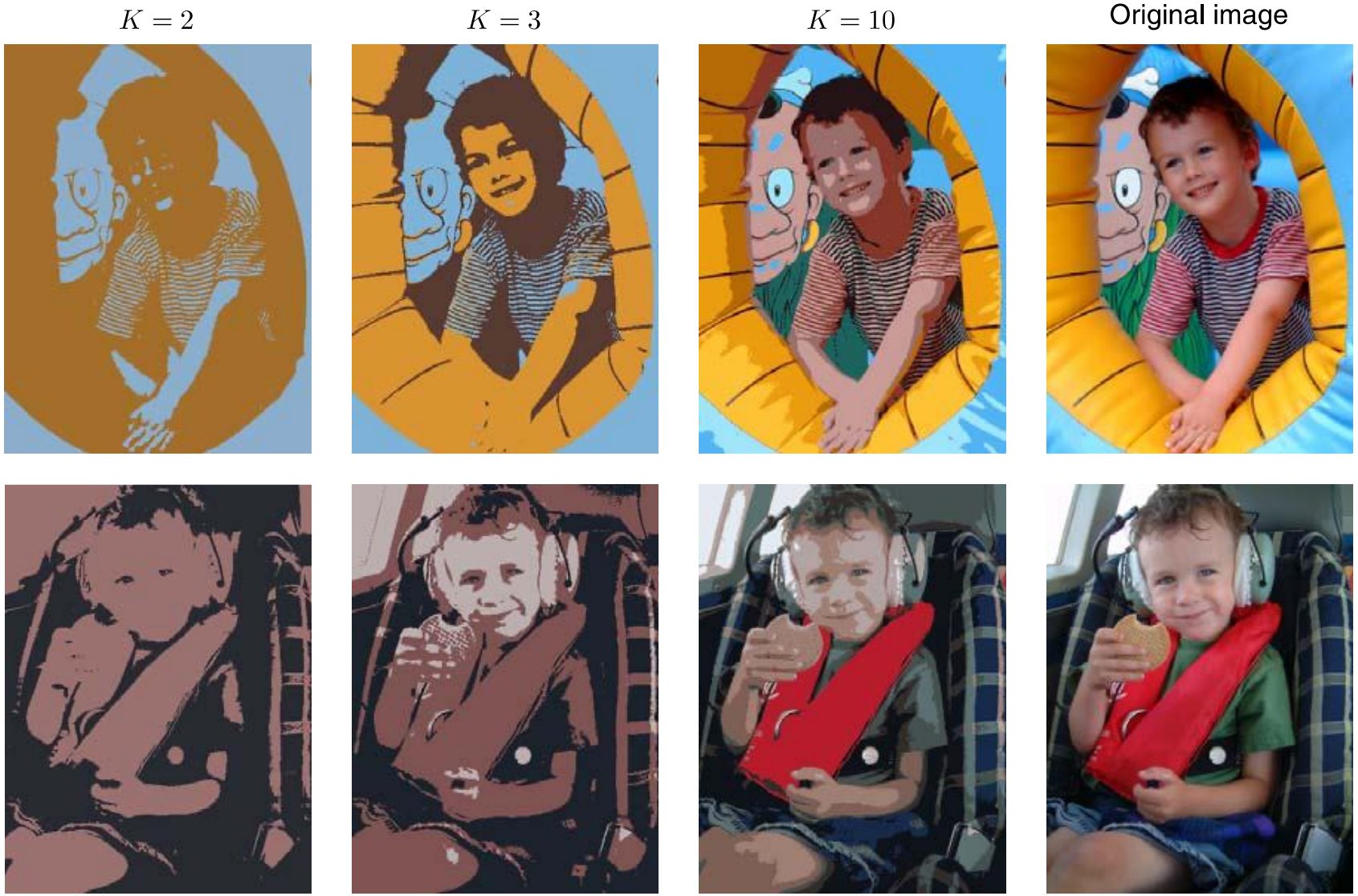
Dissimilarity

- Euclidean distance
  - $\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2$
  - Inappropriate if $\boldsymbol{x}_n$ represents catagorical data
  - Not robust to outliers
- General dissimilarity measure
  - Some dissimilarity function: $\Psi(\boldsymbol{x}_n, \boldsymbol{\mu}_k)$
  - $J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \Psi(\boldsymbol{x}_n, \boldsymbol{\mu}_k)$
  - This is called “K-medoids” algorithm
  - E-step: Fix $\boldsymbol{\mu}_k$. Assign $\boldsymbol{x}_n$ to a $\boldsymbol{\mu}_k$ with the smallest $\Psi(\boldsymbol{x}_n, \boldsymbol{\mu}_k)$
  - M-step: restrict $\boldsymbol{\mu}_k$ to one of $\boldsymbol{x}_n$ in the cluster

Hard vs. Soft assignment

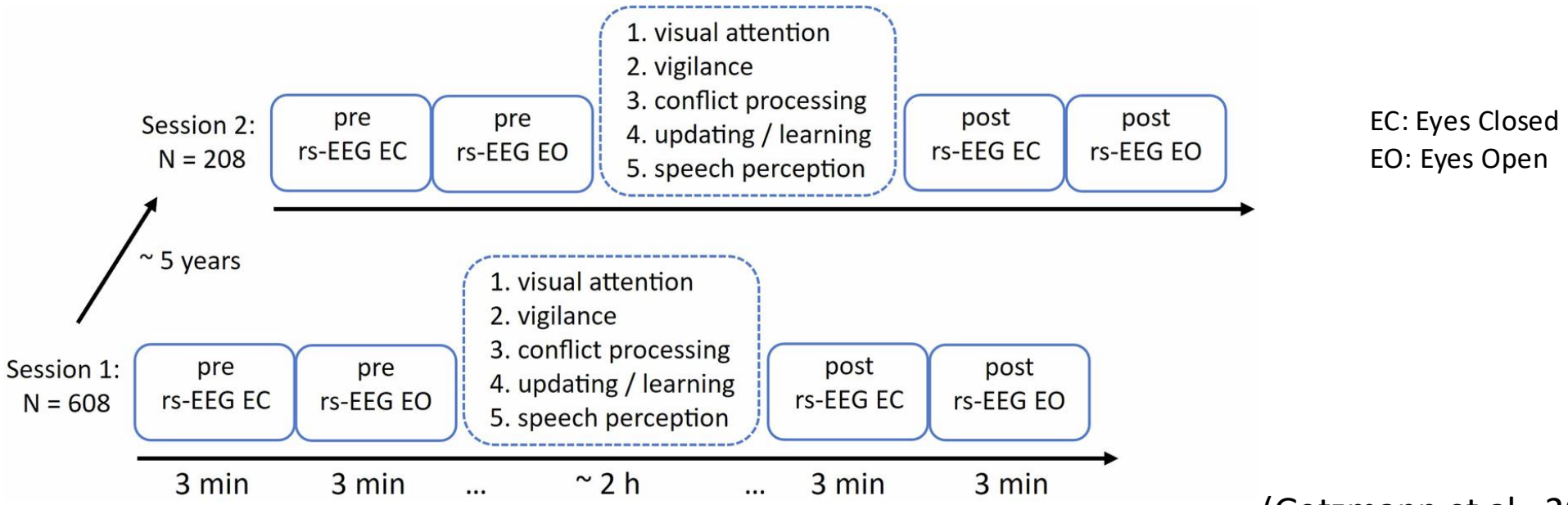
- Hard: Assign each $\boldsymbol{x}_n$ to only one cluster
- Soft: Assign each $\boldsymbol{x}_n$ to all clusters with different probablities
- Soft assignment can carry uncertainty

Example: image segmentation



Case Study: EEG microstate

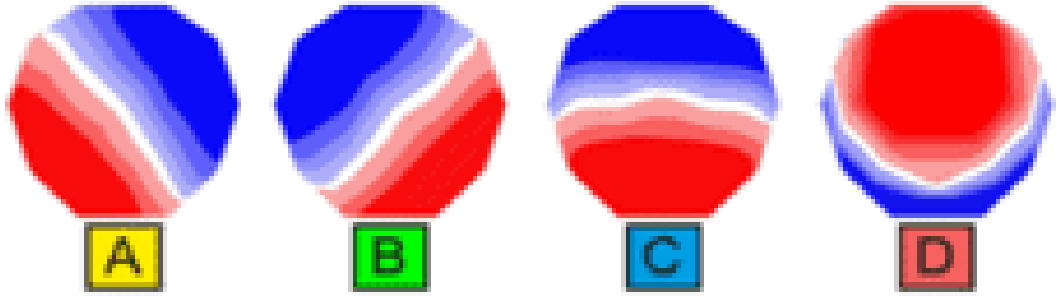
- Resting-state EEG (rs-EEG)
  - Provides a natural neurophysiological baseline
  - Potential biomarkers for clinical assessments: depression, ADHD, MCI, etc.
  - Track brain state changes before and after tasks



(Getzmann et al., 2024)

Case Study: EEG microstate

- Microstate analysis
  - Analyzes temporal dynamics of brain-wide networks
  - Examines spontaneous brain activity
  - Infers self-organizing mental states
  - Assumes that spontaneous brain activity is organized in < ms periods - the brain's overall mode of information integration → "micro"state
  - A small but prototypical set of brain microstates can describe mental states: 4 microstate maps can explain ~70% of total variation



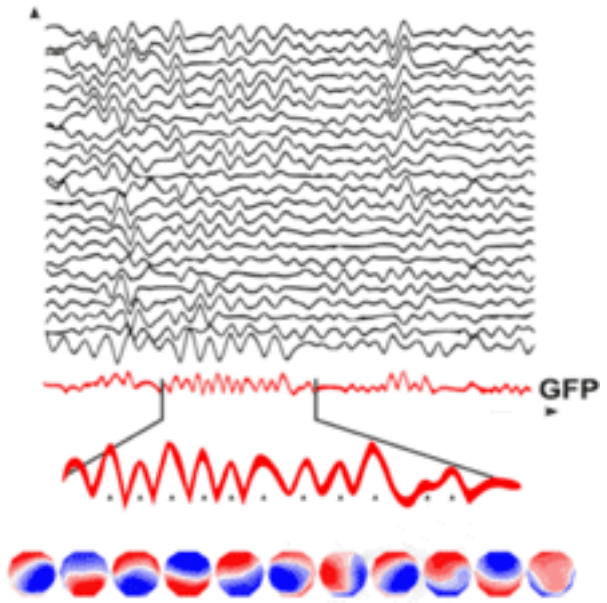
(Sapien Labs)

Case Study: EEG microstate

- Global field power (GFP)
  - Instantaneous spatial standard deviation

$$GFP(t) = \sqrt{\frac{1}{C}\left(\sum_{c=1}^{C}\left(V_c(t) - \overline{V(t)}\right)^2\right)}$$

  - EEG field potentials over the brain remain most stable at GFP peaks

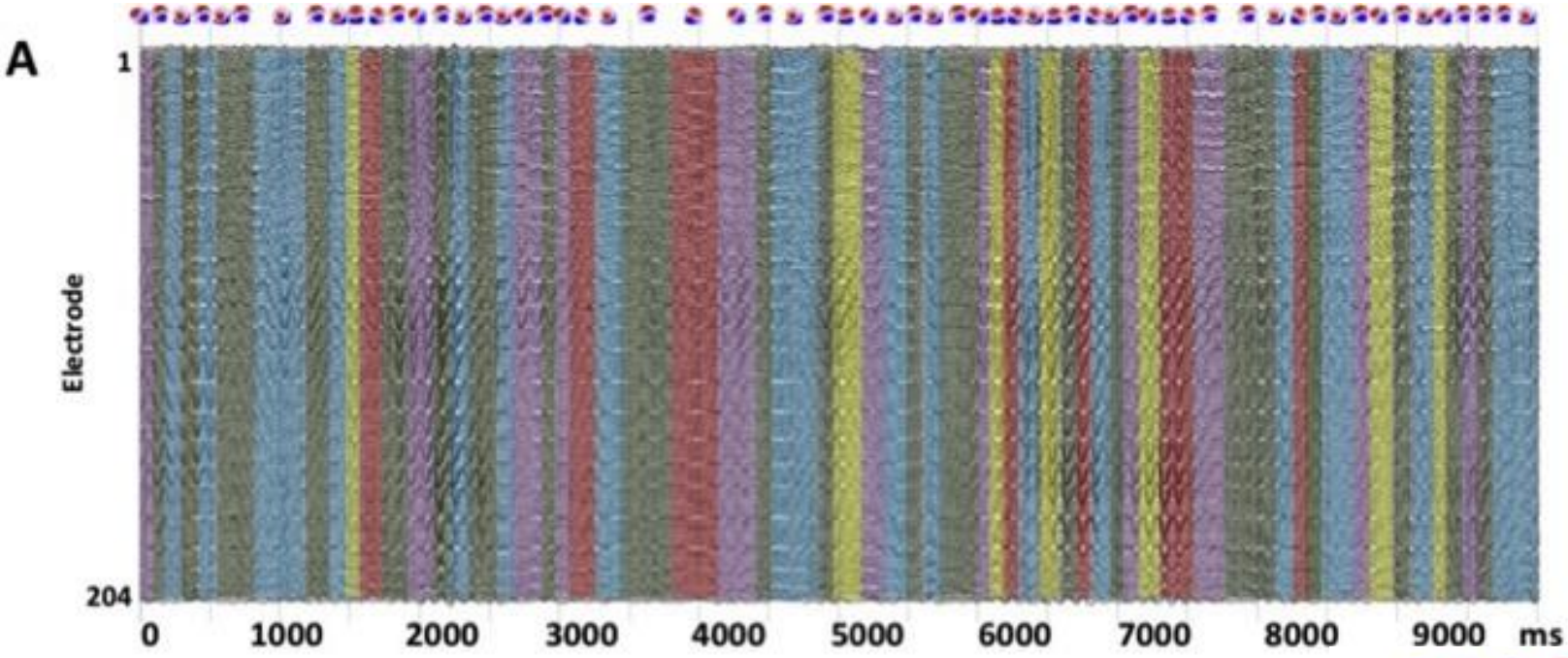


  - Referece is important to determine GFP peaks

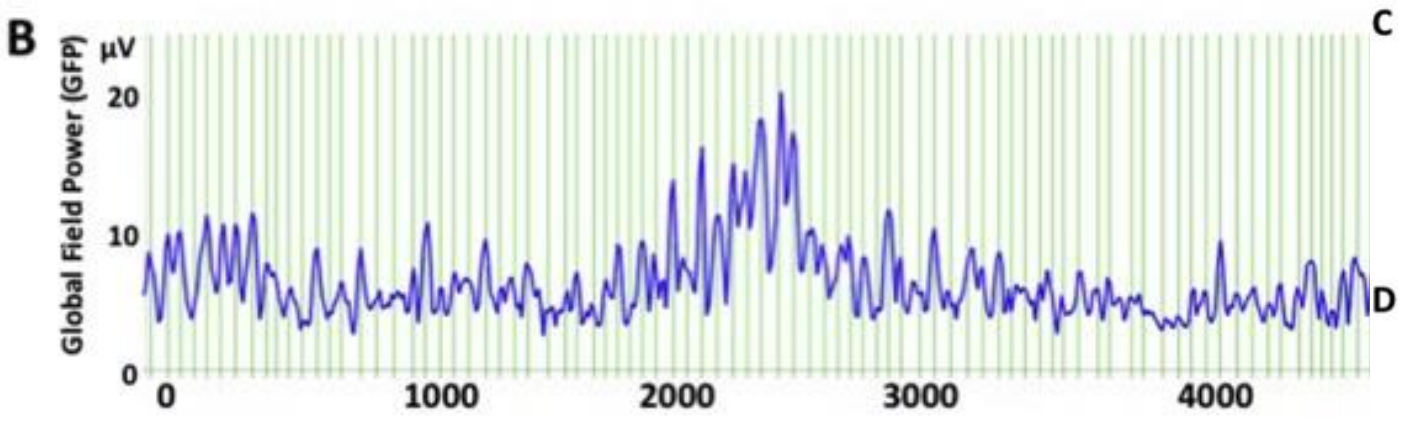
(Sapien Labs)

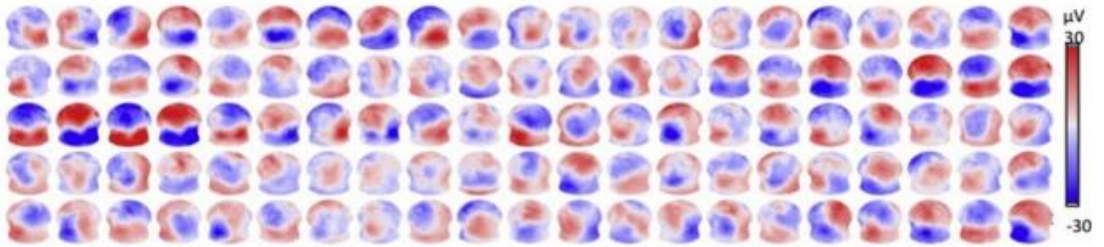
Clustering Analysis

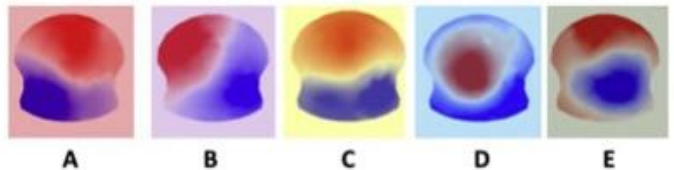
(Michael and Koenig, 2018)

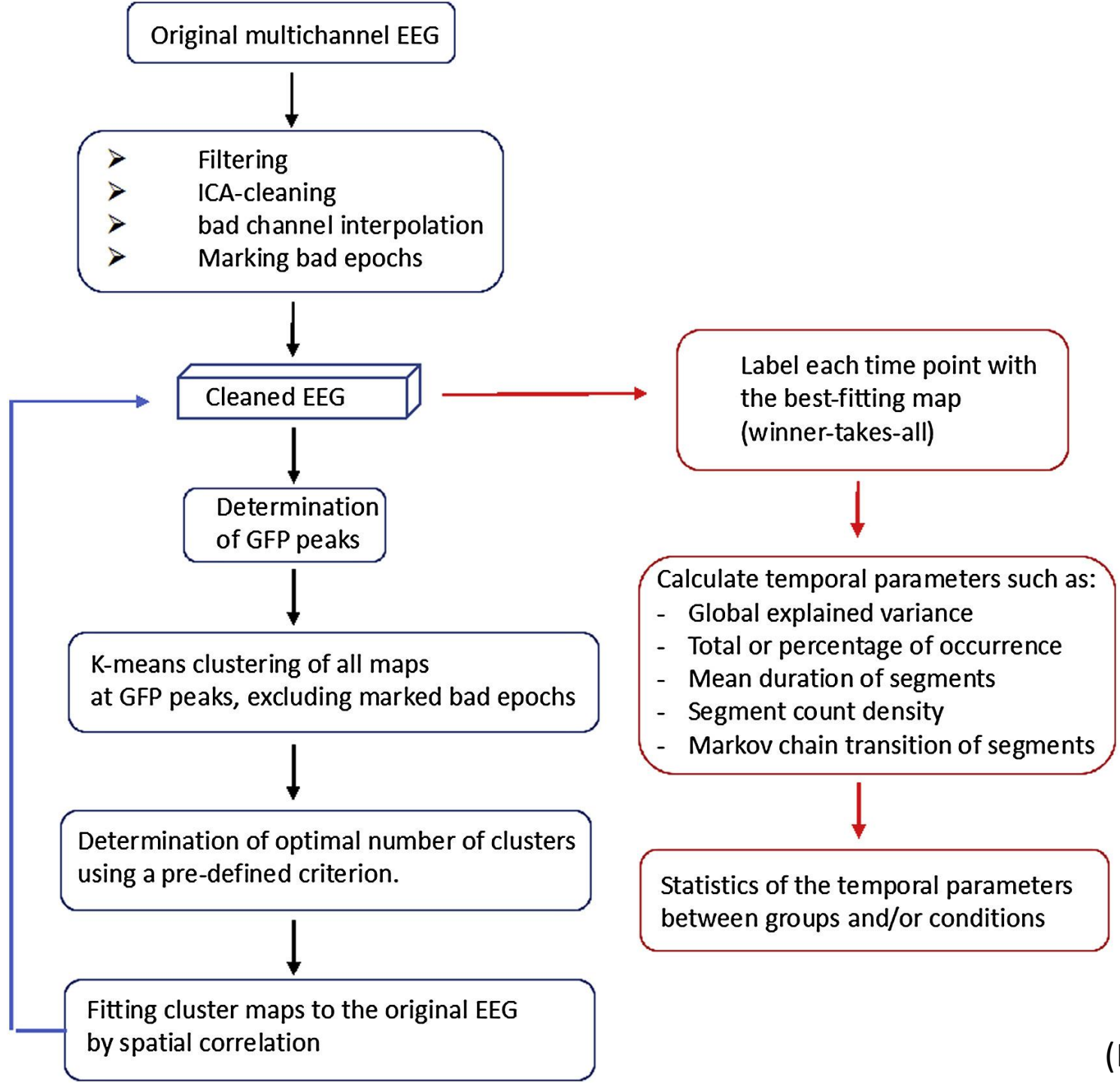


Spatial k-means algorithm





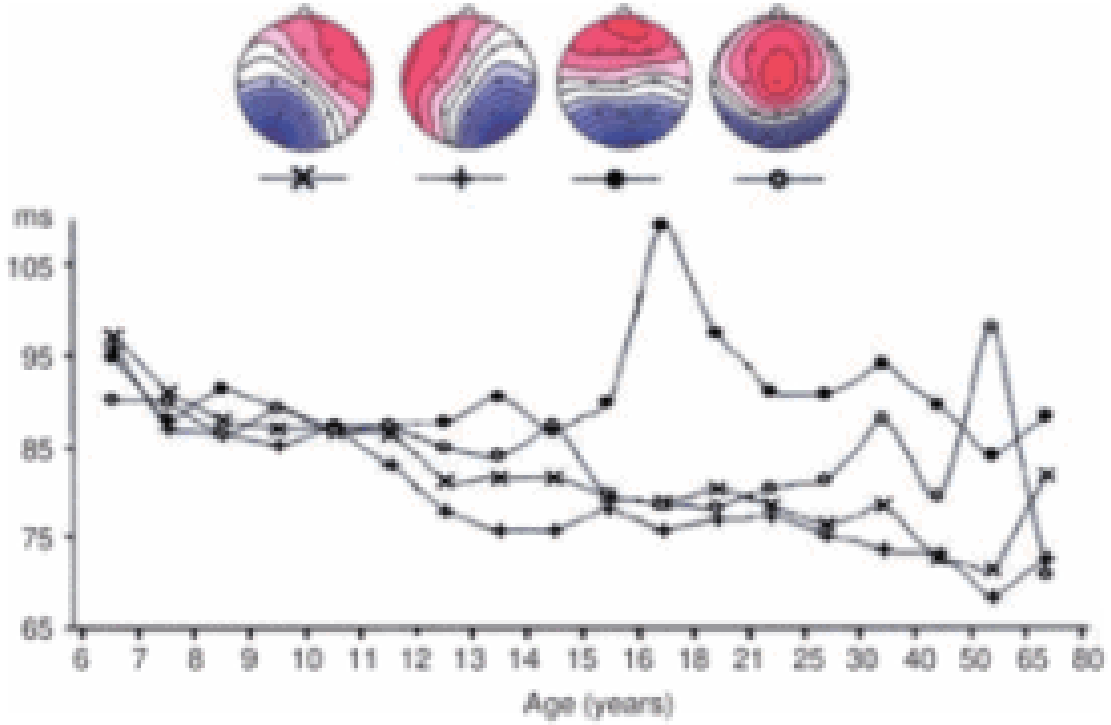




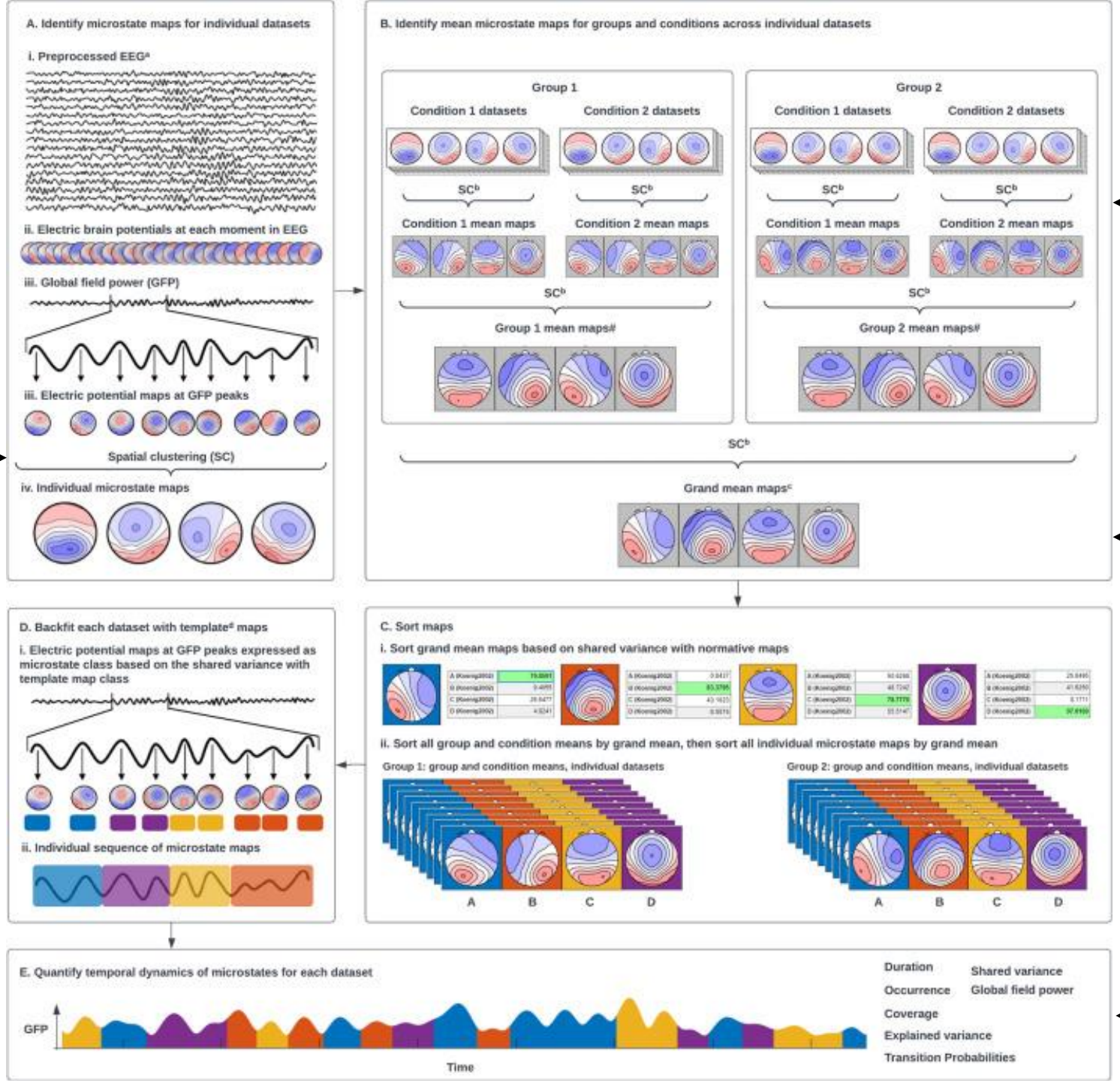
(Michael and Koenig, 2018)

Case Study: EEG microstate

- Normative EEG microstates
  - Analyze resting-state EEG
  - N = 496 subjects
  - Aging 6- 80 years old
  - Found dominant 4 microstate maps (lasting ~ 80-100 ms)



(Koenig et al., 2002)



Polarity invariant

2nd level spatial clustering

Every grand mean map should be assigned to exactly one individual map

Microstate dynamics features

(Kalburgi et al., 2024)



- (Duration) Average duration of a state X (ms)
- (Occurrence) Number of times a state X occurs (1/s)
- (Coverage) % of total time a state X occupies
- (Explained variance) % total variance explained by a state X
- (Transition probability) Probability that a state X changes to a state Y
- (Shared variance) Global explained variance explained by all states
- (Global field power) Mean GFP of all time periods assigned to X

(Kalburgi et al., 2024)